# AISE4010

## Week 4

## Convolutional Neural Networks & Deep Learning

Oct 7, 2026

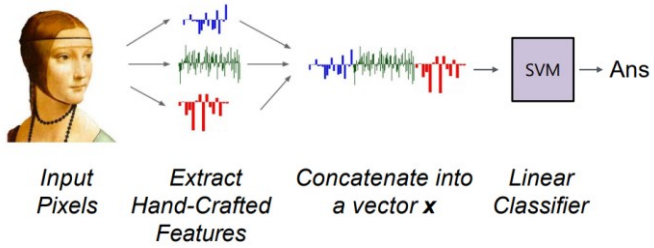
# 4.1. Deep Learning

## Convolutional Neural Networks (CNNs)

# Why Deep Learning?

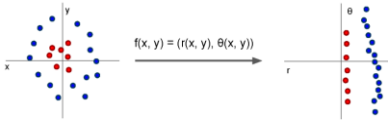
Before deep learning:

- A non-automated crucial step in a machine learning workflow: feature engineering.
- Feature extraction can not be updated during training.

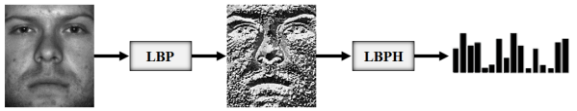



## Feature extraction in traditional ML

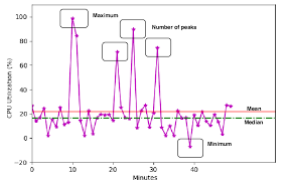
- Transforming raw, complex data into a simpler, more informative format (features).
- It improve separability between classes/patterns by emphasizing the most relevant features, addresses curse of dimensionality in ML, and removes noise and redundancy



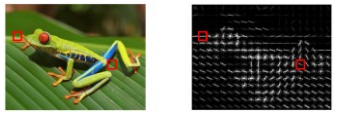

- Texture histogram: what are the local intensity differences?




- Statistical characteristics?

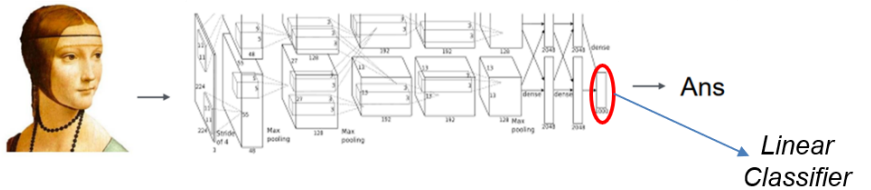



- Histogram of orientated gradients: what are the local edge information?



With Deep learning

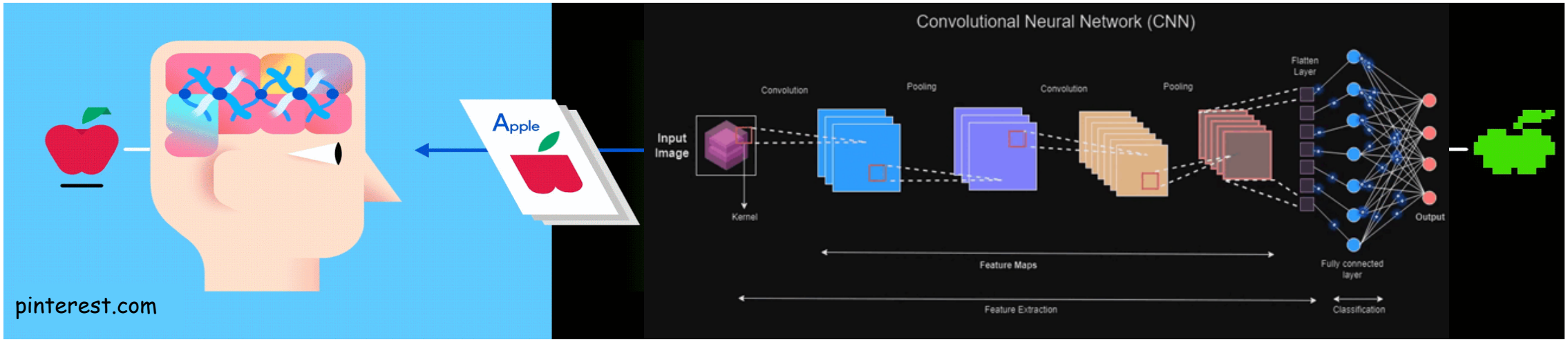
- An end-to-end neural network (no human intervention)



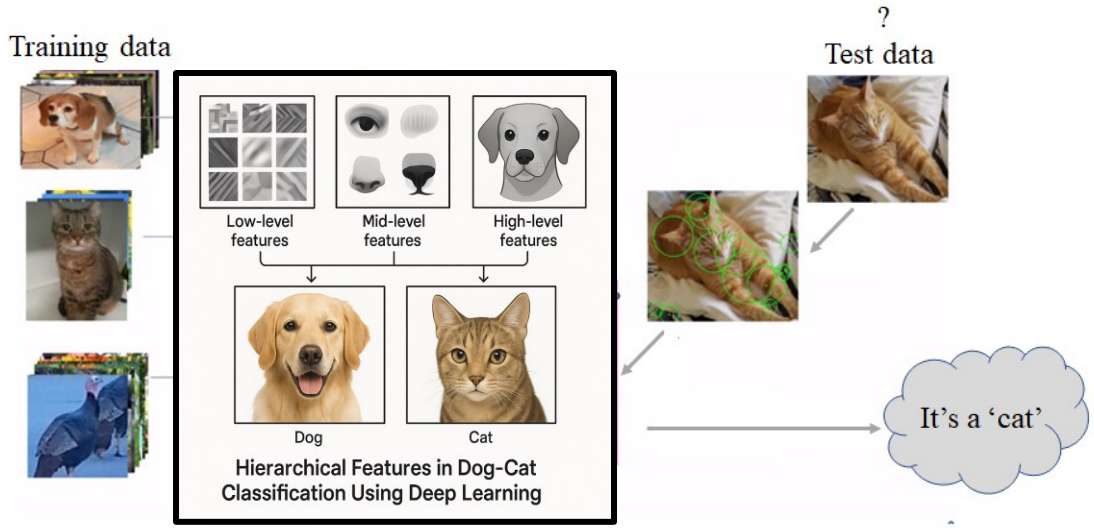

[Karpathy 2016]

# Deep Learning

- Learns features by example: if a set of input data presented to it repeatedly, gradually acquires ability to recognize patterns




- Deep learning (DL) makes problem-solving easier, because it completely automates feature engineering.
- More accuracy compared to conventional ML. Also, in DL the whole network is updated during training.
- There are several variants of deep learning such as Convolutional Neural Networks, Deep Belief Network, Deep Boltzmann Machines, Recurrent Neural Networks and Transformers.

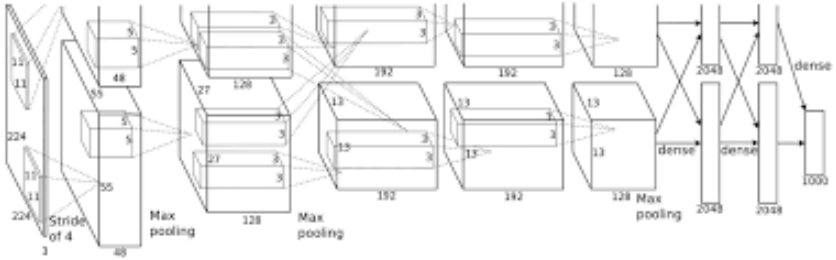


# Major Breakthrough of CNNs in 2012

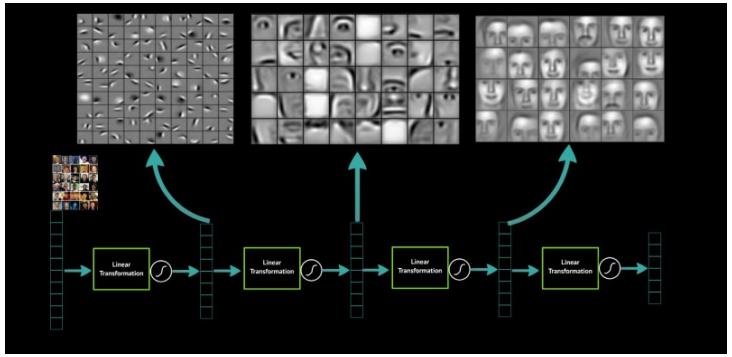




- Geoffrey Hinton: The "godfather of deep learning," co-developed the backpropagation algorithm in 1986, which became crucial for training deep networks.
- CNN were well known in the computer vision and ML communities following the introduction of LeNet by Yann LeCun et al., who pioneered convolutional neural networks in 1989.
- AlexNet (Alex Krizhevsky, Ilya Sutskever and Ph.D. advisor Geoffrey Hinton) was introduced in UofT and won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012.




- Key Advantages of CNNs for Time Series Forecasting:
  - Local connectivity: employ convolutional layers to focus on local regions of the input, which enables them to capture short-term patterns effectively, which is crucial in time series forecasting.
  - Hierarchical representation: multiple convolutional layers, to learn hierarchical representations of data, that capturs both short-term and long-term dependencies.
  - Parallel processing of multiple sequences simultaneously, efficient for large datasets and multivariate time series.



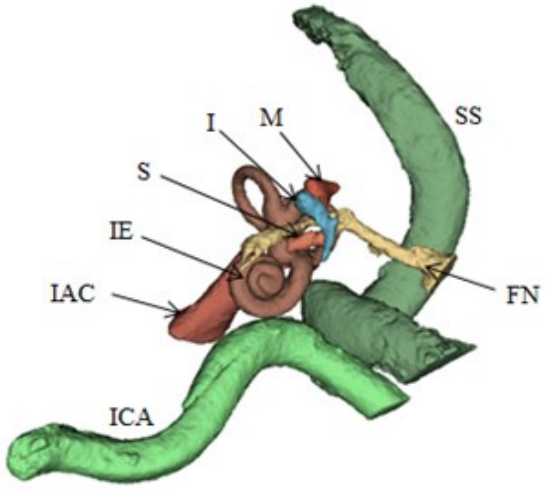
# Interesting & Complicated Applications

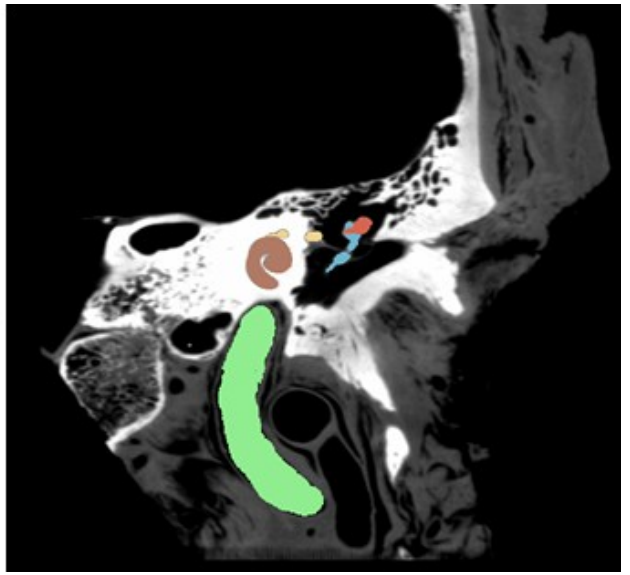
- **Semantic Segmentation**

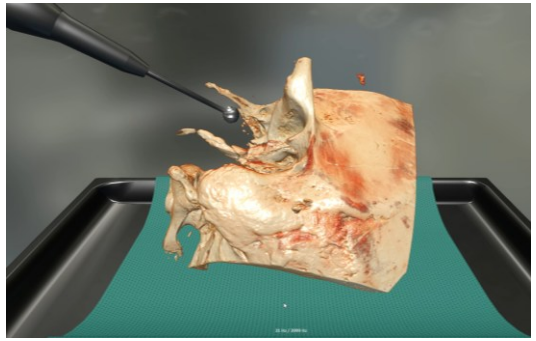
Driving Scene



Medical Images

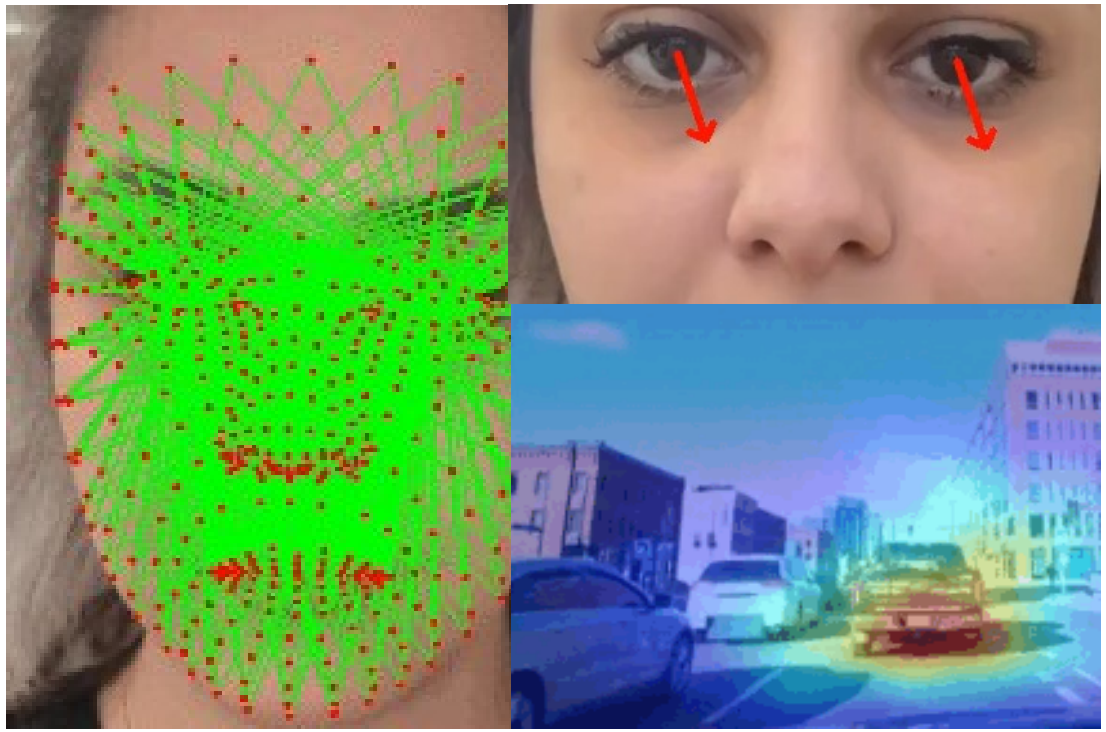




# Interesting & Complicated Applications

- **Pose Estimation, Gaze Estimation**



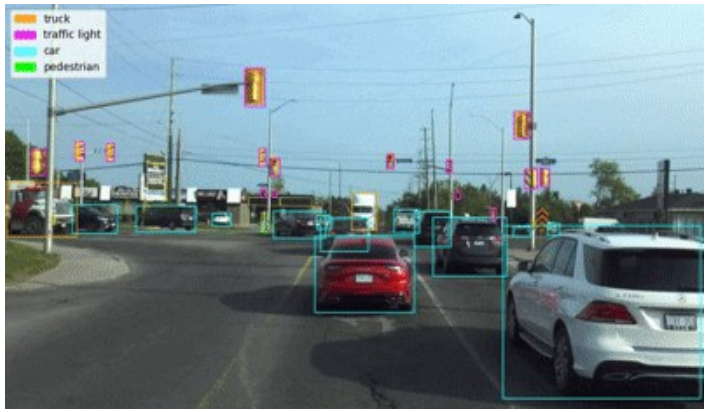


# Interesting & Complicated Applications

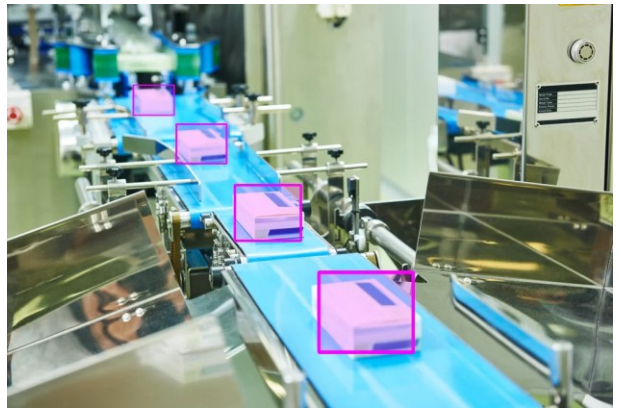
- **Object detection**




Perception in Autonomous Driving



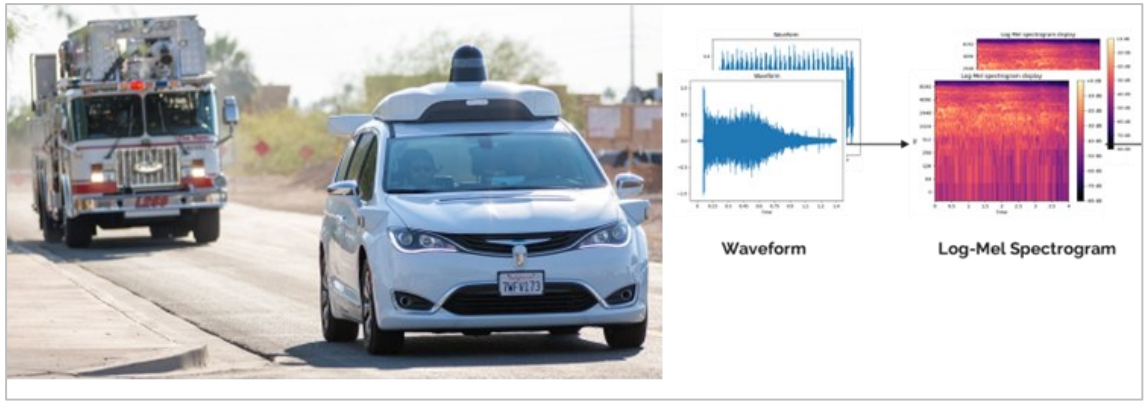

Perception in Autonomous Driving



Manufacturing

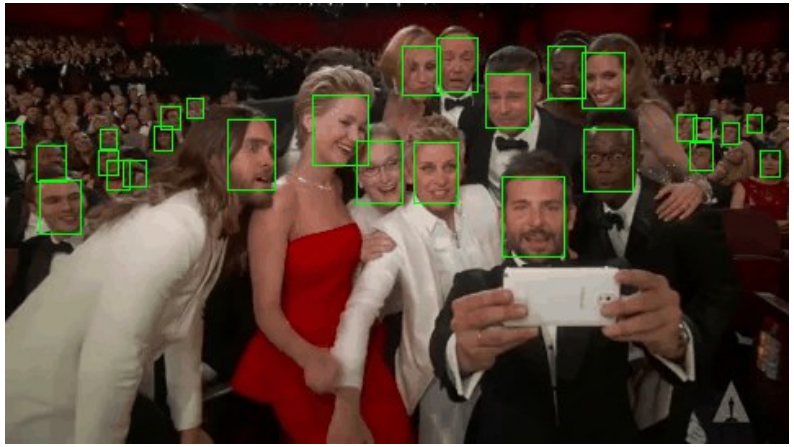
# Interesting & Complicated Applications

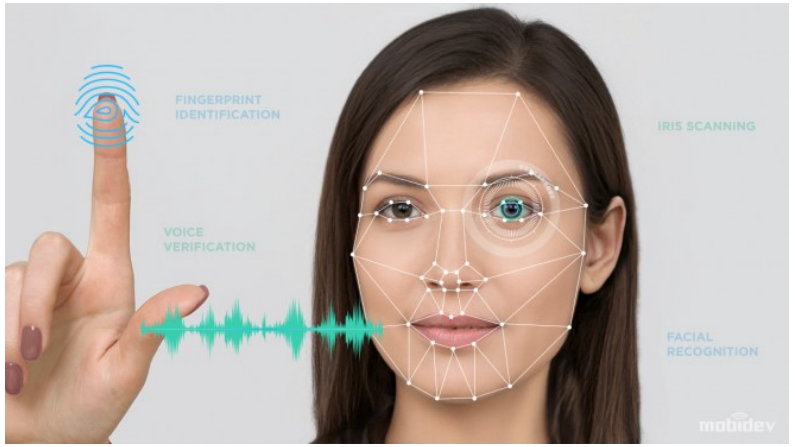
- Sound/Voice Recognition






- Face detection
- Identifications (face/iris/fingerprint/voice recognition)

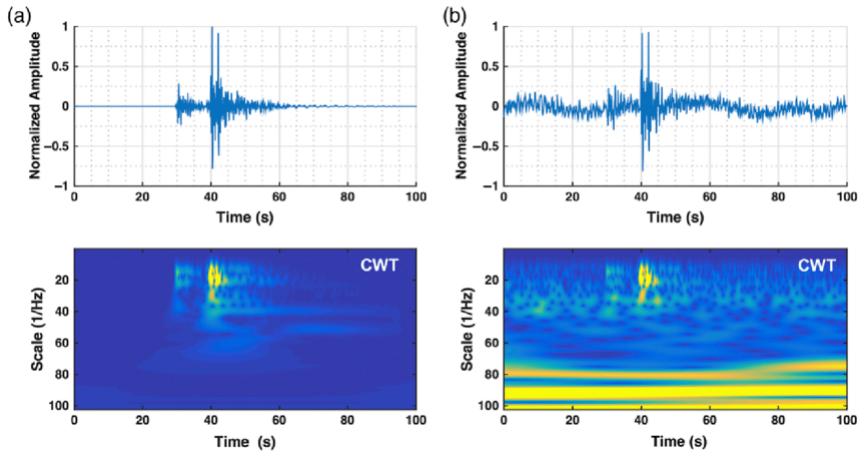




# Interesting & Complicated Applications

- **Synthetic data generation**



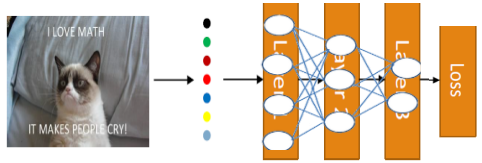



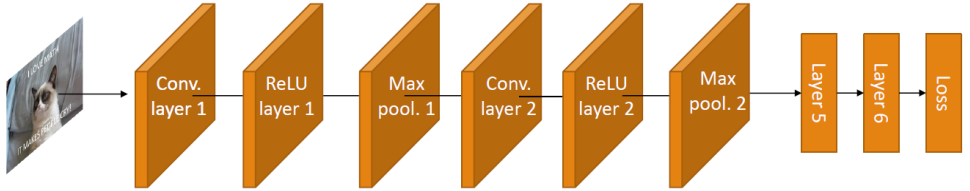


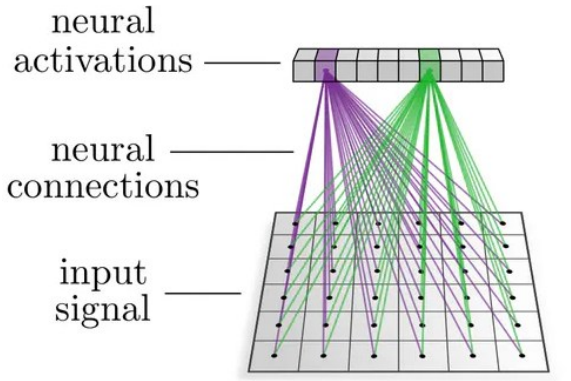

# Convolutional Neural Network (CNN) vs Multi-Layer Perceptron (MLP)

Fully Connected Layer vs Locally Connected Layer

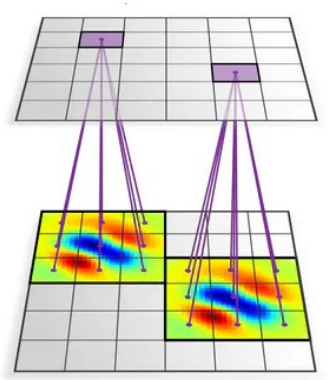
- MLP's parameters scale based on the size of the input image and the number of neurons, a CNN's parameters scale based on the size of the filters and the number of feature channels
- CNN uses special Convolution and Pooling Layers whereas MLP is just a network of fully connected Neurons
- CNN is generally used for huge or bulky data as compared to MLP



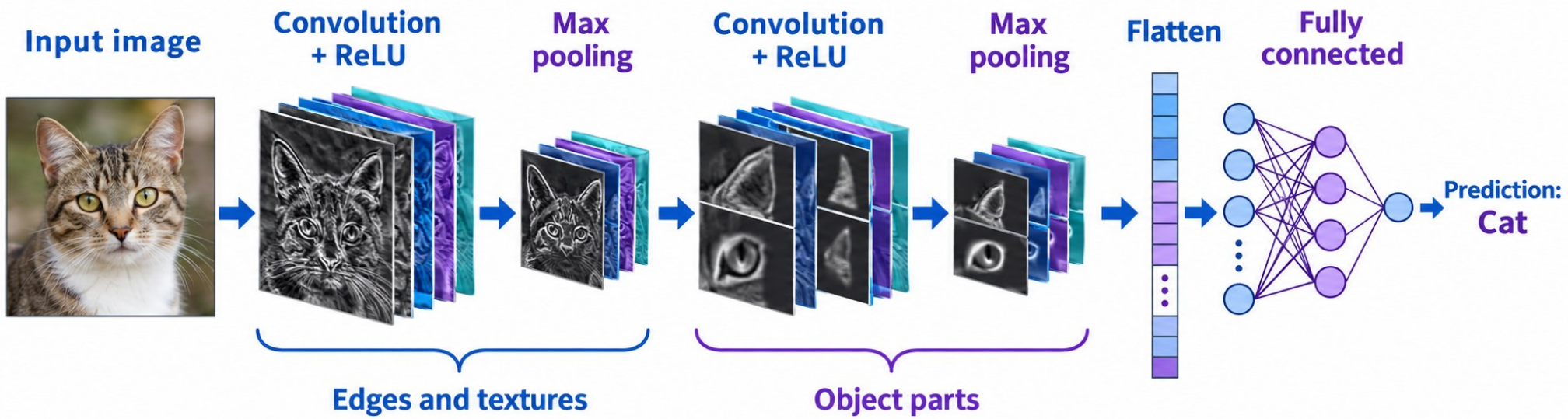

fully connected (MLP)



convolutional (CNN)

# CNN Architecture

- Layers: Lower layers capture basic features, while deeper layers identify more complex patterns like parts of objects or entire objects.
- Learning Process: CNNs learn the filters during training. The network adjusts the filters to minimize the loss between the predicted and actual outcomes ≡ optimizing the feature extraction process.
- Pooling Layers: After convolution operations, pooling (e.g., Max pooling) reduces spatial size of representation. A pooling layer downsamples spatial dimensions of the input feature maps while preserving important information.
- Activation Functions: introduces non-linearities. This helps the model learn more complex patterns.



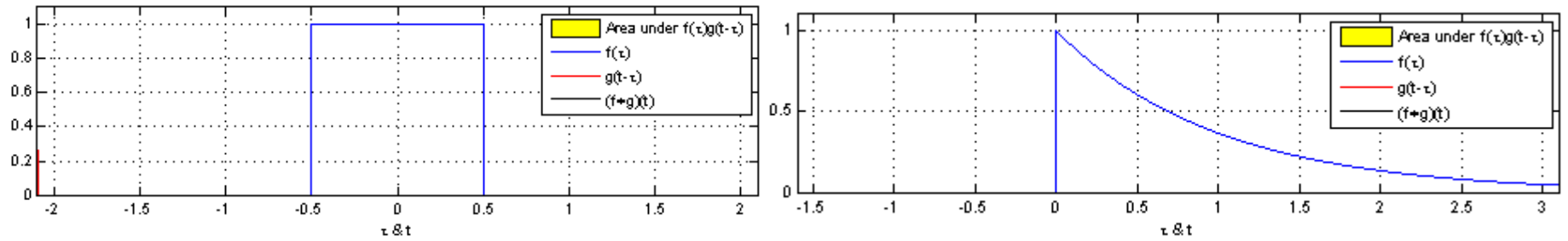

Filters are learned during training

# 4.2. Convolution and Its Different Types

# Convolution Operation

The convolution of two functions $f$ and $g$ is denoted by $*$ as the integral of the product of the two functions after one is reversed and shifted

$$(f * g)(t) \stackrel{\text{def}}{=} \int_{-\infty}^{\infty} f(\tau)g(t-\tau)\,d\tau = \int_{-\infty}^{\infty} f(t-\tau)g(\tau)\,d\tau$$




A mathematical operation where a matrix (or kernel) traverses the input and performs a dot product with the overlapping region. A convolution operation involves the following steps:

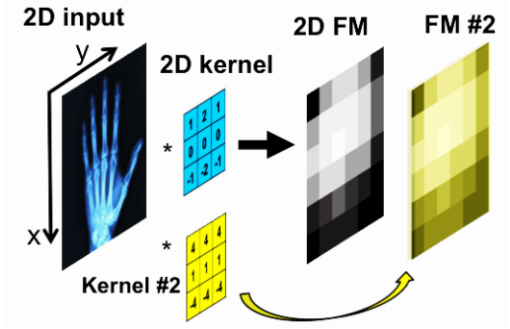
- Define a small matrix (filter).
- This kernel moves across the input matrix.
- At each location, convolution operation computes the dot product of kernel and the portion of input it overlaps.
- The result of each dot product forms a new matrix, that represents transformed features of the original input.

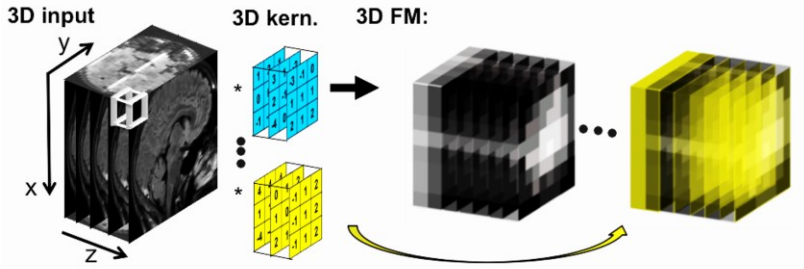
# Filter/Kernel Dimension - 1D vs. 2D vs. 3D CNN

- Spatial dimensions (height and width): refers to the filter's receptive field over the input. Common spatial dimensions include 3x3, 5x5, or 7x7. For example, a 3x3 filter processes a 3x3 window of the input at each step. We usually use odd numbers for filter size so that they are applied symmetrically around our input data

- Depth (number of channels): must match the number of channels in the input volume it is convolving over. If the input is an RGB image, it has 3 channels, so the filter will also have a depth of 3. If the filter is applied to a feature map generated by a previous layer with 64 channels, filter's depth will also be 64.

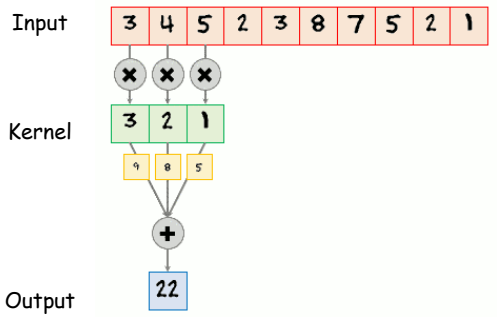


# Convolution Operation – 1D Convolution

- In Conv1D:
  - kernel/filter slides along the input data (one dimension).
  - performing element-wise multiplication followed by a sum.




```
import numpy as np

np.convolve([3, 4, 5, 2, 3, 8, 7, 5, 2, 1],
            [1, 2, 3],
            mode='valid')
```

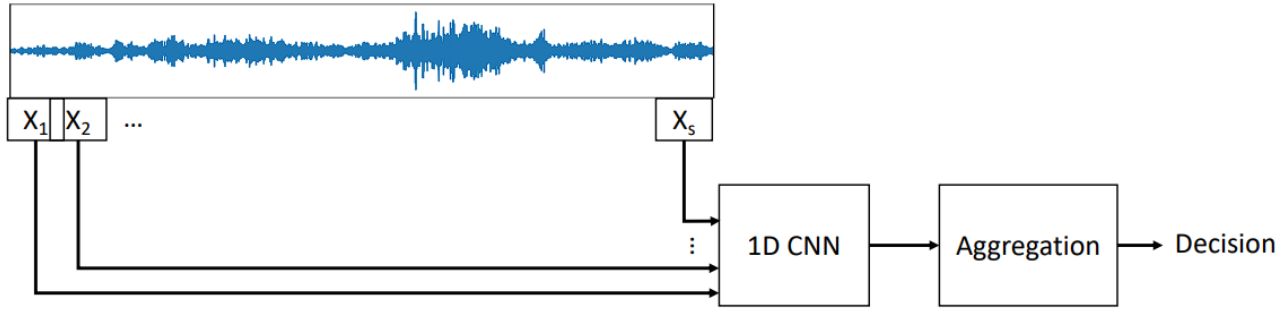
```
array([22, 24, 22, 20, 32, 43, 33, 20])
```

- Applications: (any data that requires kernel sliding in only one dimension):
  - Natural Language Processing (text classification, text generation, language modelling)
  - Time-Series Analysis/Forecasting (stock market prediction, weather forecasting, cyber security, traffic prediction)
  - Signal Processing (low/high pass filtering, smoothing for ECG, EEG, EMG, etc.)
  - Genomics and Bioinformatics (gene sequence analysis, mutation detection, protein structure prediction)



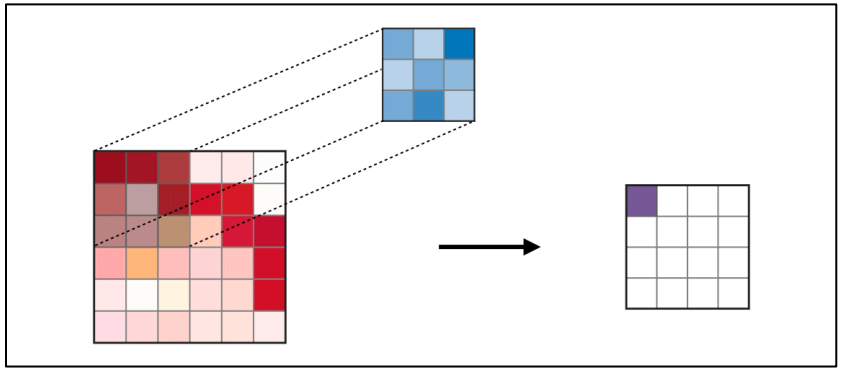


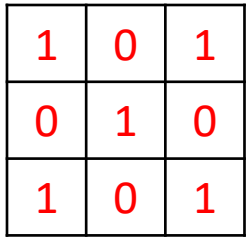




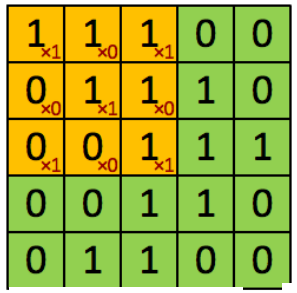
# Convolution Operation – 2D Convolution

- Overlay Kernel on Image: Start from top-left corner of image and so that its center aligns with current image pixel.
- Element-wise Multiplication: Multiply each element of kernel with the corresponding element of image it covers.
- Summation: Sum up all products from the element-wise multiplication, which forms a single pixel in output feature map.
- Continue the Process: Slide kernel over to the next pixel and repeat the process across entire image.

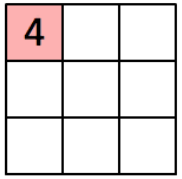




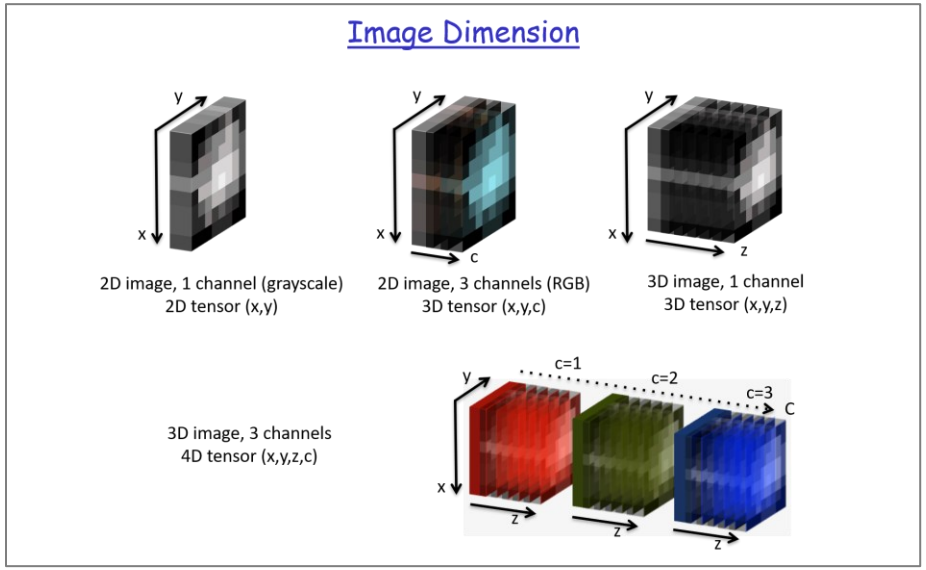

Kernel

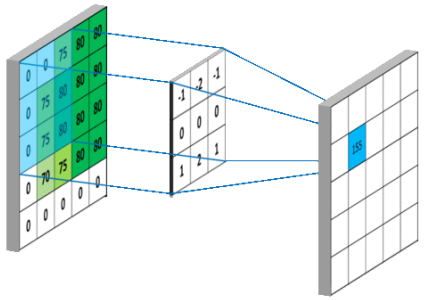



Image



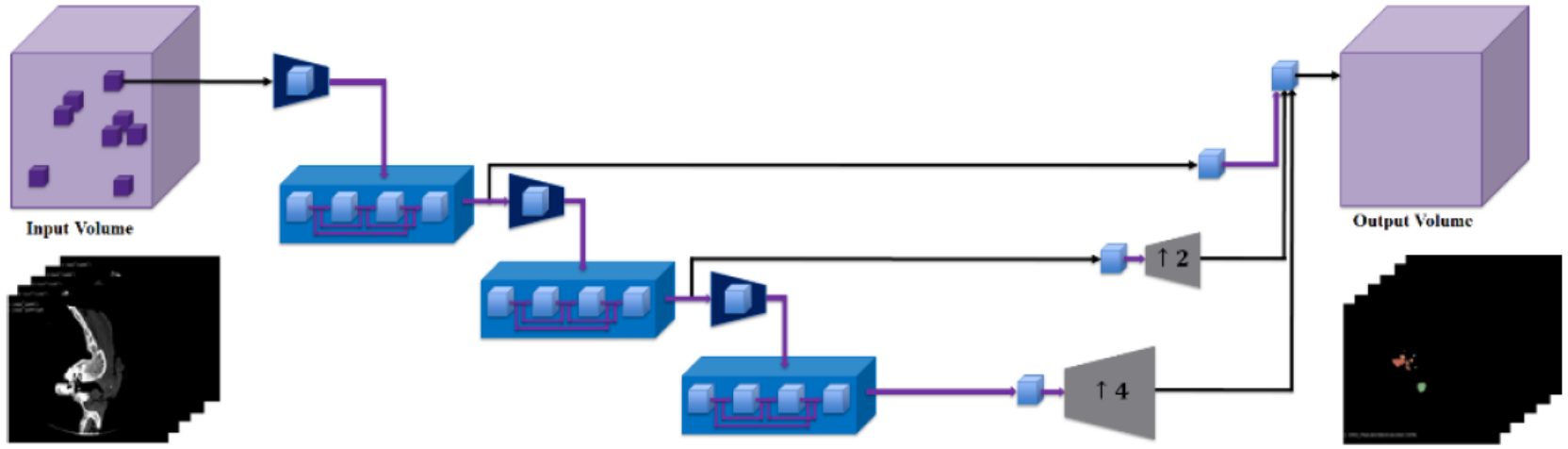

Convolved Feature



# Convolution Operation – 3D Convolution

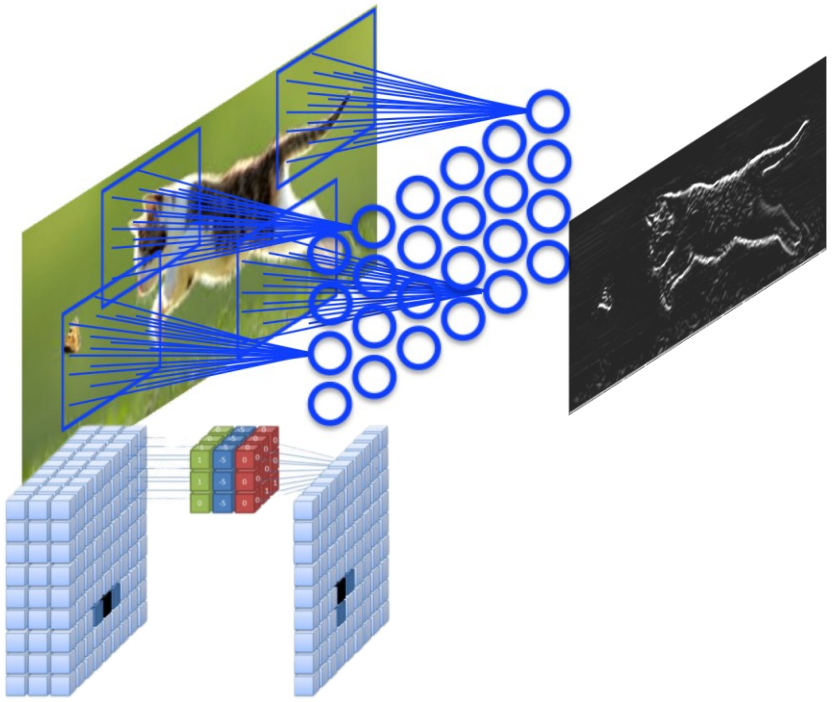
- 3D convolution extends the concept of 2D convolution by adding dimension, which is useful for analyzing volumetric data.
- Like 2D convolution, a three-dimensional kernel moves across the data, but it now simultaneously processes three axes (height, width, and depth).
- Applications: Video Analysis (width, height, time), Medical Imaging (3D scans, such as MRI or CT scans).



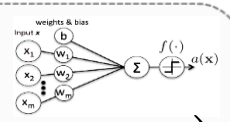
# 4.3. CNN Layers and Characteristics

# Recap - Weight Sharing in CNN – Feature Maps

- Each neuron in hidden layer connects to a small region of input (local receptive field of hidden neuron)
- Hidden neurons learn an overall bias and each connection learns a weight (to analyze its receptive field )
- Multiple neurons of a feature map (FM), share the same kernels (filters), and share same weights and bias
- All neurons of a FM look for same pattern/feature, at different locations (⇒ translation invariance)
- Activations of a FM's neurons (FM/Activation map) is computed by convolving image with FM's kernel!
- Convolution of a filter of size $F \times F \times C$ and input of size $I \times I \times C$ gives a feature map (activation map) of size $O \times O \times 1$.
- A FM can be perceived as an image (the result of convolution).



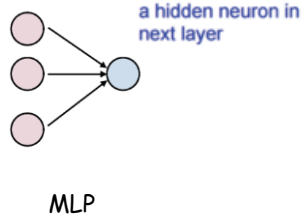
***Single neuron***:



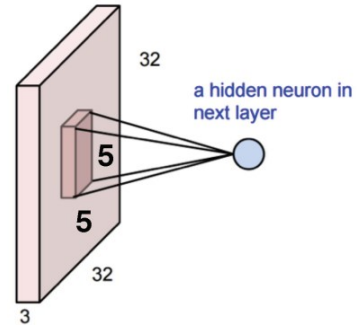
$$a(\mathbf{x}) = f\left(\sum_i w_i x_i + b\right)$$

**Feature Map:**

$$\mathbf{a}(\mathbf{x}) = f(\mathbf{x} * \mathbf{w} + b)$$



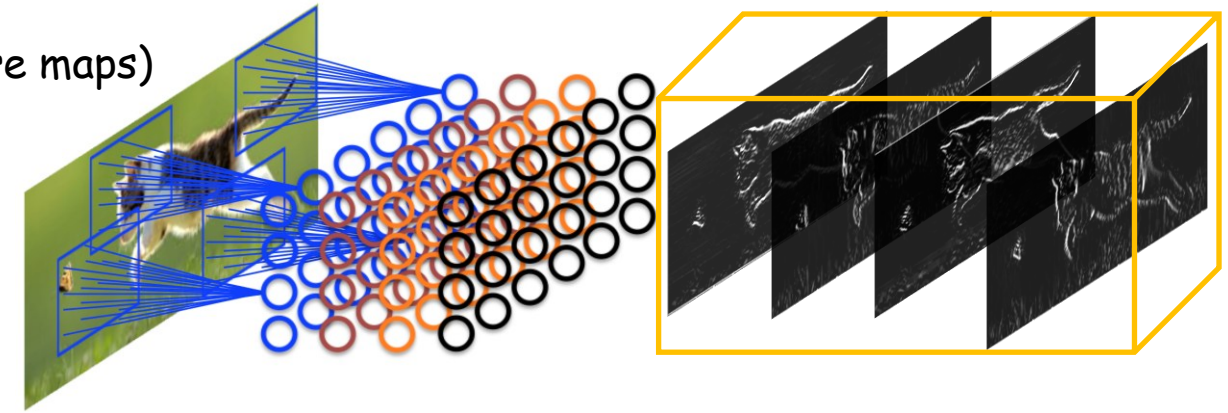

MLP



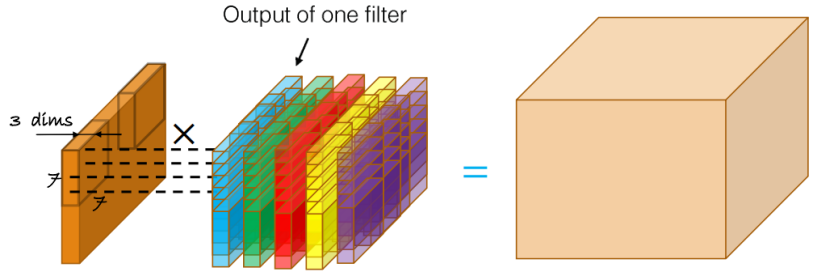

CNN

# Convolutional Layers

- We can detect multiple features per layer (multiple feature maps)
- Multiple filters produce multiple feature maps.
- A layer can be perceived as a multi-channel image



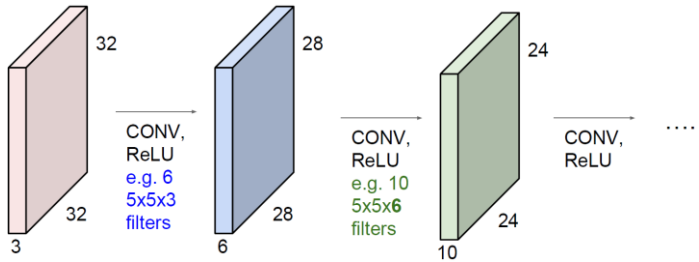
Example. For each FMof size 5×5×3 we need 5×5×3=75 shared weights, plus a single shared bias. Totally 76 parameters to learn.
If we have 20 feature maps that's a total of 20×76=1520 parameters

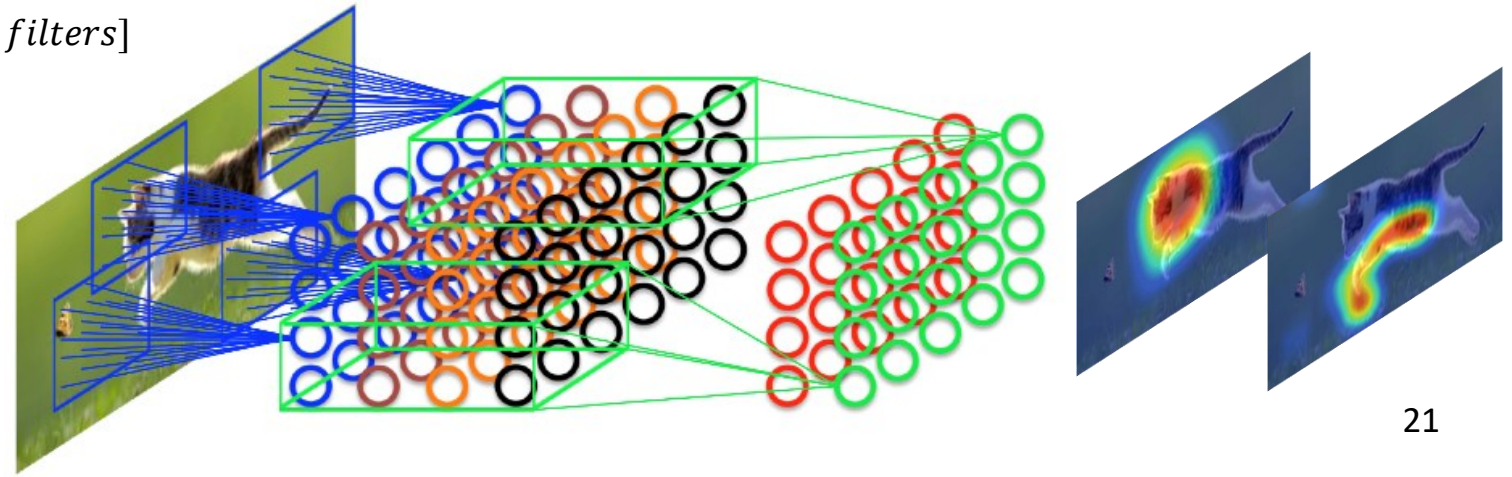



## Deep CNN - Multiple Convolutional Layers

- Sequence of convolution layers
- Deeper layers process FMs (channels) of previous layers to extract more complex representations

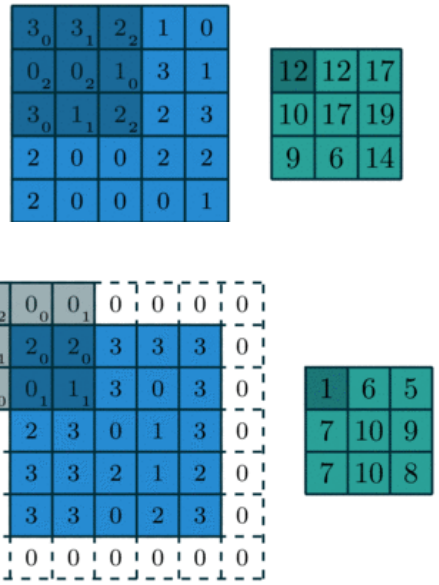
$$Output\ dimension = \left[\frac{N-F}{stride}+1\right] \times \left[\frac{N-F}{stride}+1\right] \times [\#\ of\ filters]$$

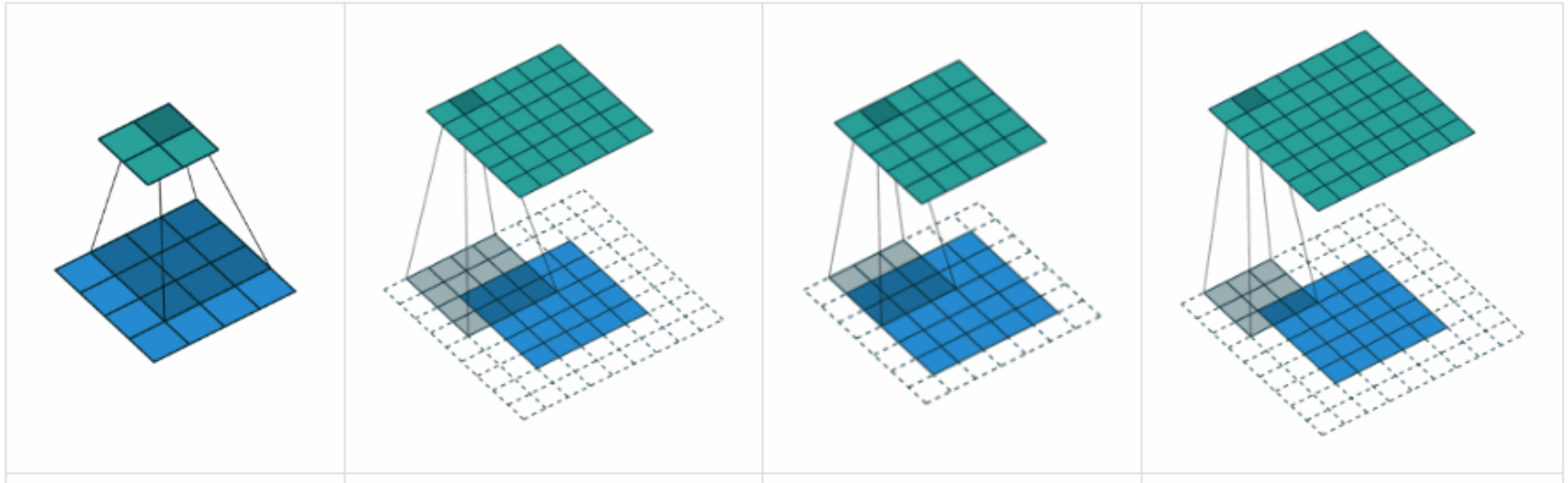


# Padding

- The inputs at the corner are less counted than those in the middle. This means that the inputs don't get the same amount of weights.
- If we just keep applying the convolution, we might lose the data too fast.
- Padding is the trick we can use here to fix this problem. Also, it makes our output array the same size as our input array, if necessary. As its name, padding means giving additional data at the boundary.
- does a filter always have to move one pixel at a time? Of course not. We can also make it move two steps or three steps at a time both in the horizontal and vertical ways. This is called 'stride.'



# Padding

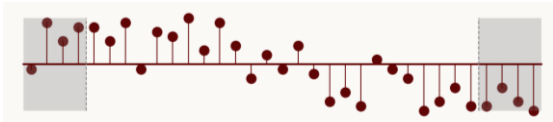
- Padding in 1D

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Input | 5 | 2 | 8 | 6 | 1 | 0 | 5 | 3 | 9 | 4 |
| Kernel | 3 | 2 | 6 | | | | | | | |
| Output | 58 | 46 | 63 | 38 | 21 | 19 | 63 | 48 | | |

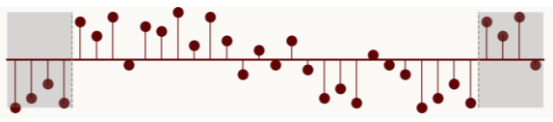
Zero Padding

| | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Input | 0 | 5 | 2 | 8 | 6 | 1 | 0 | 5 | 3 | 9 | 4 | 0 |
| Kernel | 3 | 2 | 6 | | | | | | | | | |
| Output | 16 | 58 | 46 | 63 | 38 | 21 | 19 | 63 | 48 | 62 | | |

- The downside of padding with zeros is that we are actually fabricating data.
- It's possible to corrupt our signal.
- For instance, if our one-dimensional data was a sequence of stock prices, adding zeros at the beginning and the end might make investors nervous. They don't reflect reality.
- Another option for padding that can be quite benign is padding with non-zero constant values, such as the median of the array, the average of the first few data points, the value of the first point, or mirror/circular padding.

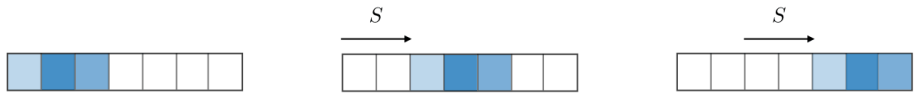


Mirror Padding

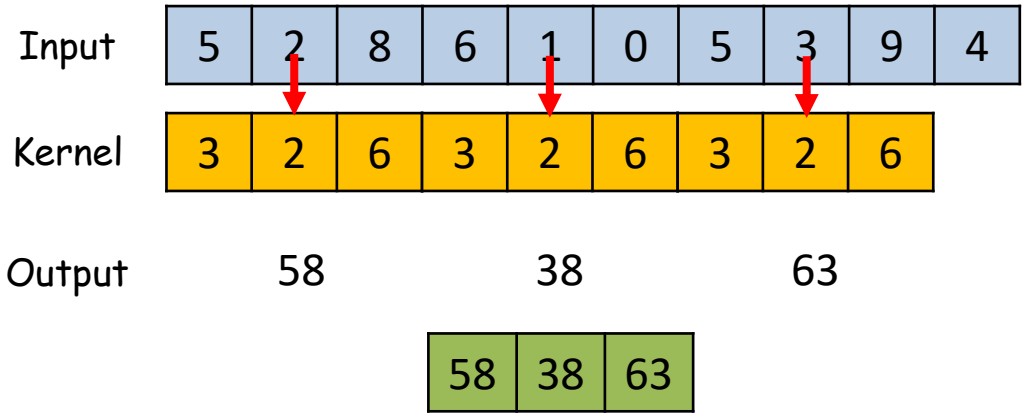


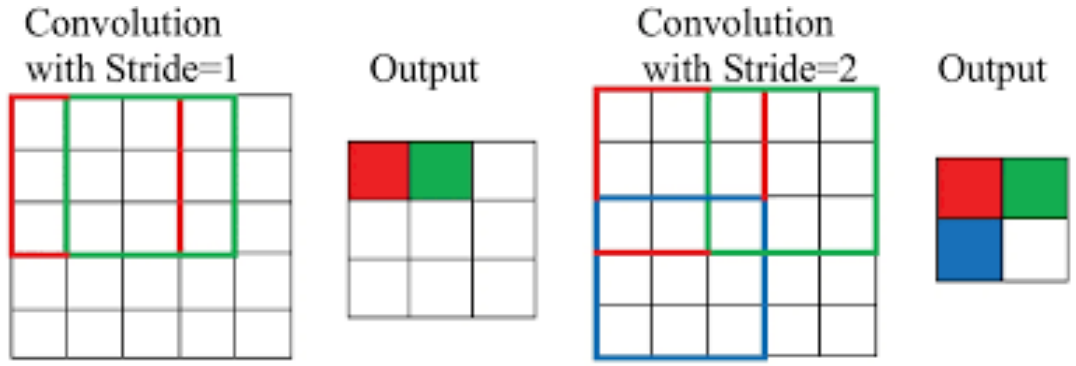
Circular Padding

# Stride



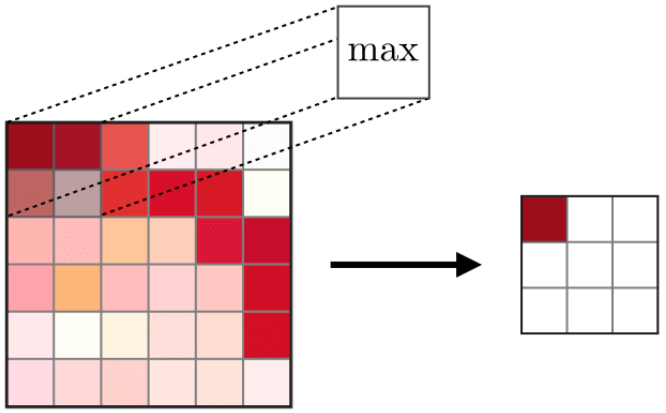

- For a convolutional or a pooling operation, the stride $S$ denotes the number of pixels by which the filter/kernel moves after each convolution operation.
- If $S = 1$, filter moves one pixel. If $S = 2$, filter moves two pixels and so on. A larger stride will result in a smaller output spatial dimension.
- It's a useful trick in calculating convolutions for neural networks more efficiently. Increasing the stride can decrease the computational load.
- A higher stride allows the network to capture broader patterns. It is beneficial when the model needs to capture more global features rather than focusing on finer details.
- Strides can be used as an alternative to pooling layers for downsampling the input.



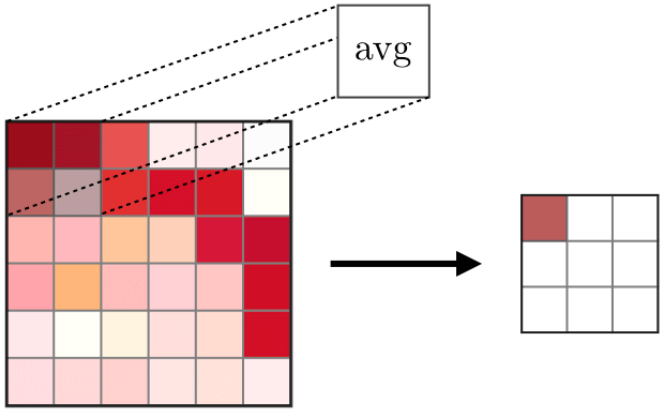



# Pooling Layer - Down Sampling Feature Maps

- Pooling layer is a downsampling operation. Aggregate multiple values into a single value.
- Typically applied after a convolution layer, which does some spatial invariance.
- Pooling works separately on each activation map.
- It acts like a mechanism to check if a particular feature exists within a certain area of the input, without keeping track of the precise location.
- Reduces the size of the layer output/input to next layer by filtering out less relevant details, reducing noise, makes computations faster and lowers the chance of overfitting.
- In particular, max and average pooling are special kinds of pooling where the maximum and average value is taken, respectively. Max is the preferred choice of pooling.



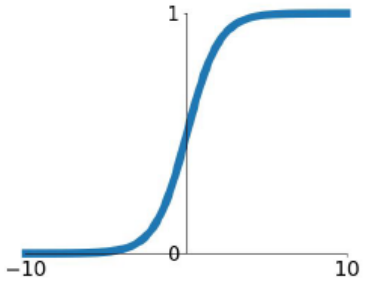

extracts only the strongest signal




smoothing away sharp details

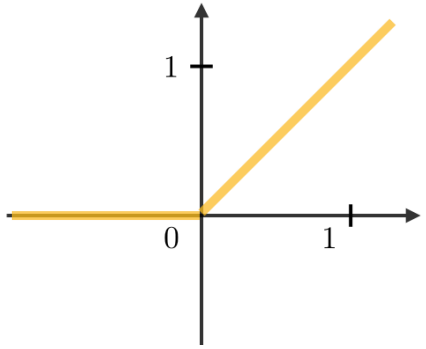
# Activation Functions (non-linearity)

Sigmoid



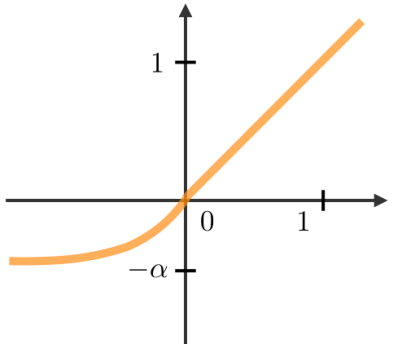

$$f(x) = \frac{1}{1 + e^{-x}}$$

ReLU



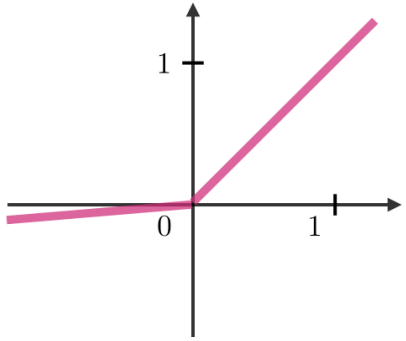

$$f(x) = \max(0, x)$$

ELU




$$f(x) = \max(\alpha(e^{x} - 1), x)$$
with $\alpha \ll 1$

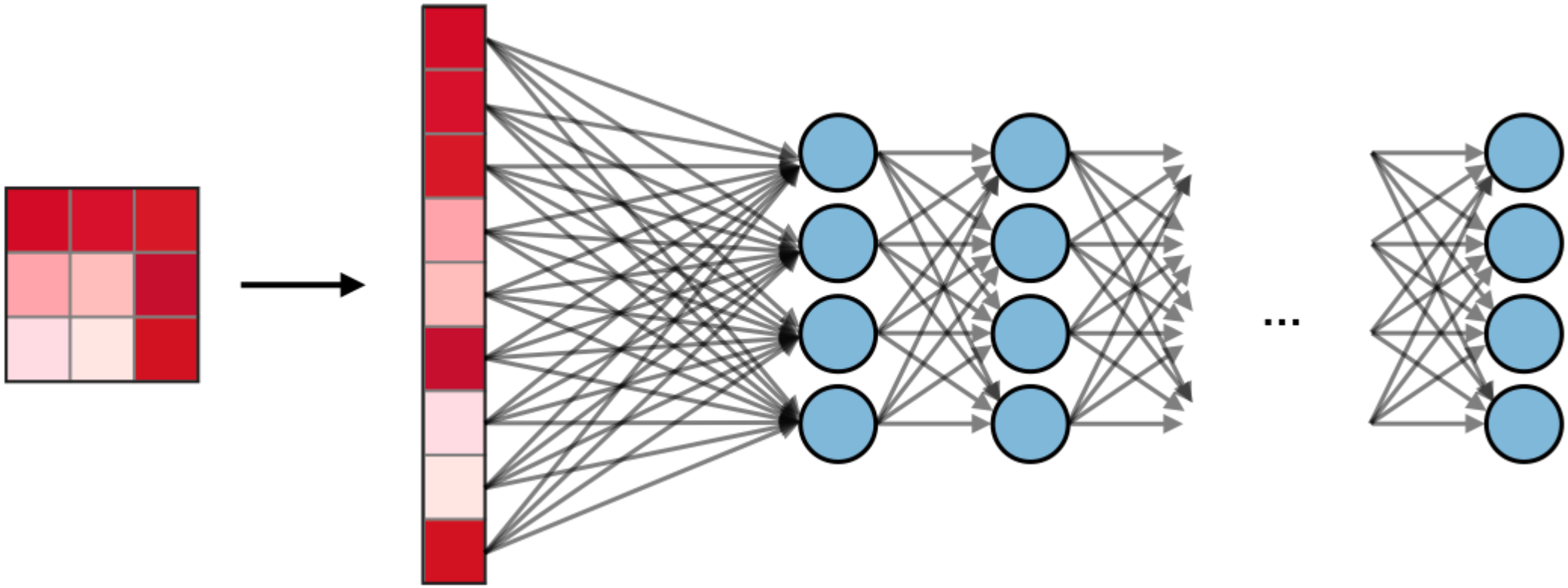
Leaky-ReLU



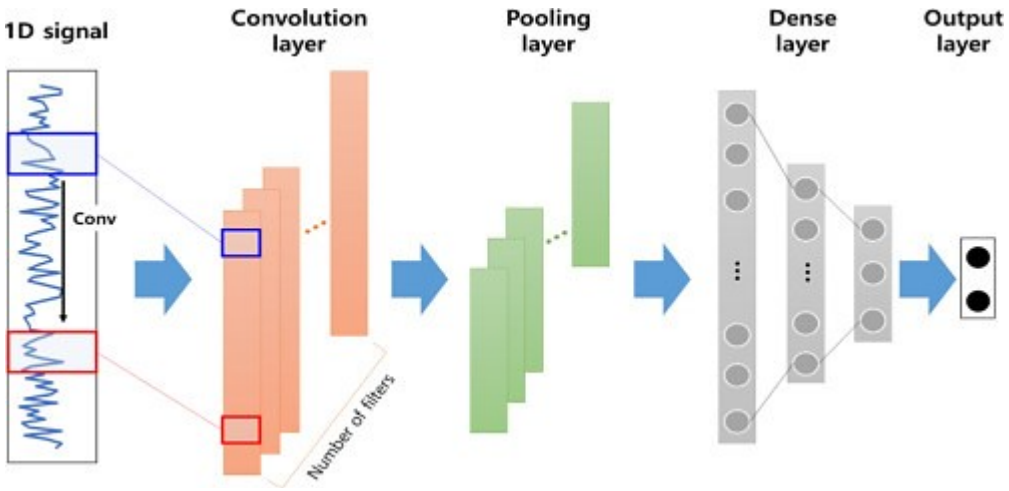

$$f(x) = \max(\epsilon x, x) \text{ with } \epsilon \ll 1$$

# Fully Connected (FC) Layers

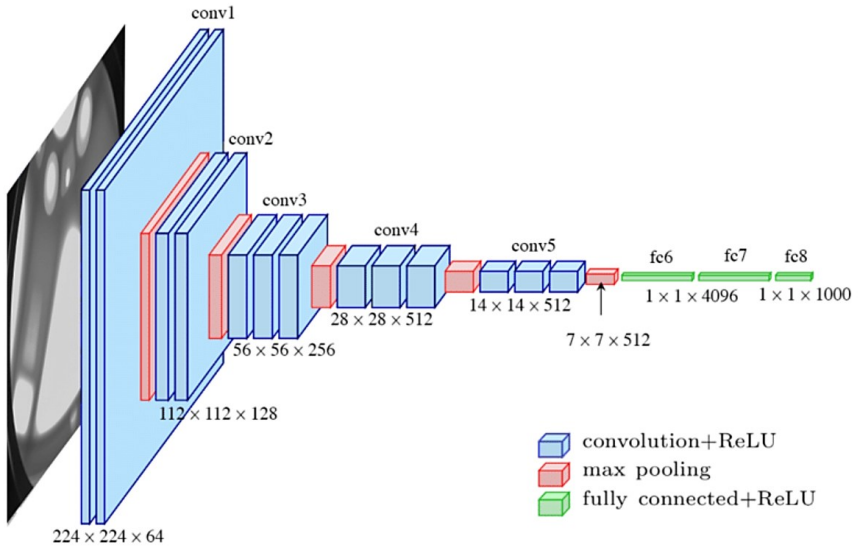
- The fully connected layer (FC) operates on a flattened input where each input is connected to all neurons.
- If present, FC layers are usually found towards the end of CNN architectures and can be used to optimize objectives such as class scores.
- FC layer throws away spatial coordinates.



# Deep CNN Architectures



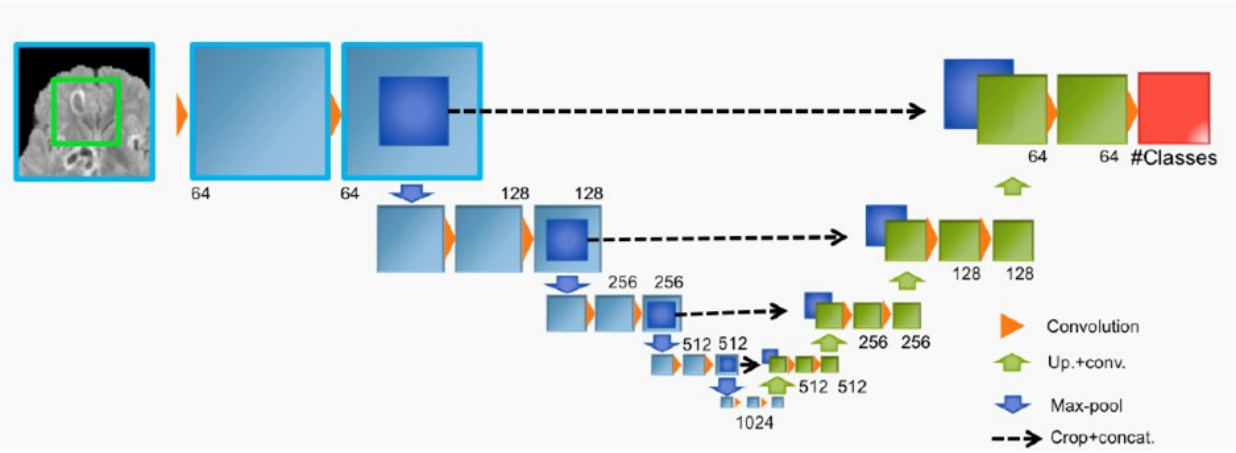

1D CNN




2D CNN

## A Different Category

- Fully Convolutional Network (FCN): Changing fully connected layers in CNN into 1x1x1 convolutional layers
- This category is only for Image Data: Learns a mapping from pixels to pixels (pixelwise-output /label-map)
- Needs upsampling (transposed convolution/deconvolution)



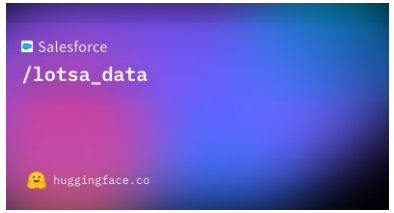

FCN

# Dataset & Computational Resources Are Critical




- **ImageNet** (and the ImageNet Large-Scale Visual Recognition Challenge/ILSVRC) has been key to training deep learning methods (J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, CVPR, 2009).

  It has >14 million images (hand-annotated) and more than 20,000 categories of objects.







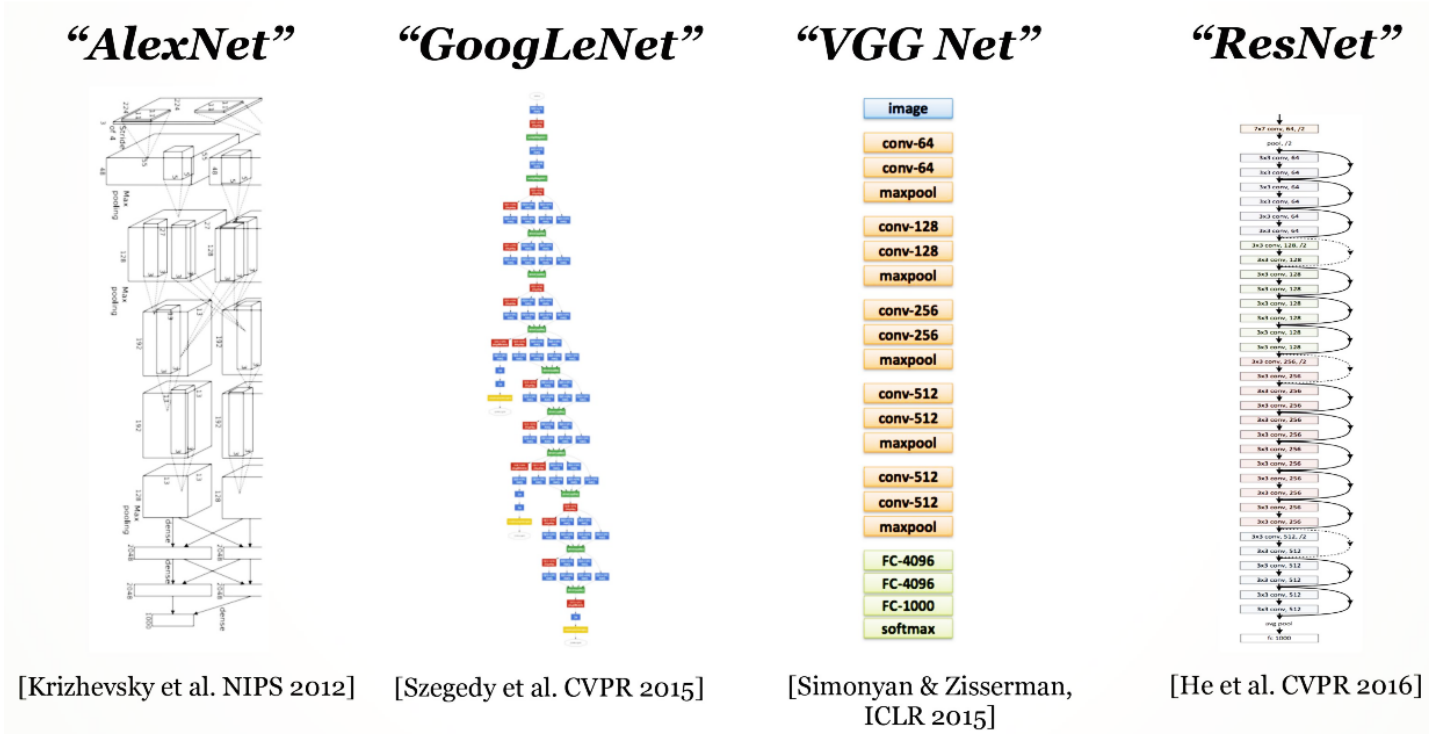
- **UTSD (Unified Time Series Dataset):** Trillions of time points aggregated across dozens of sub-repositories. Developed by Tsinghua University for training the Timer and Sundial model families.
- **LOTSA (Large-scale Open Time Series Archive):** ~27 billion time points across 500+ datasets, developed by Salesforce for training the Moirai foundation model. Includes diverse domains including energy, transport, weather, finance, sales, nature, and healthcare.

With NVIDIA GPU-accelerated deep learning frameworks, researchers and data scientists can significantly speed up deep learning training, which could otherwise take days and weeks to just hours and days.



# Taking Advantage of Popular Models

- EX. ImageNet Benchmark



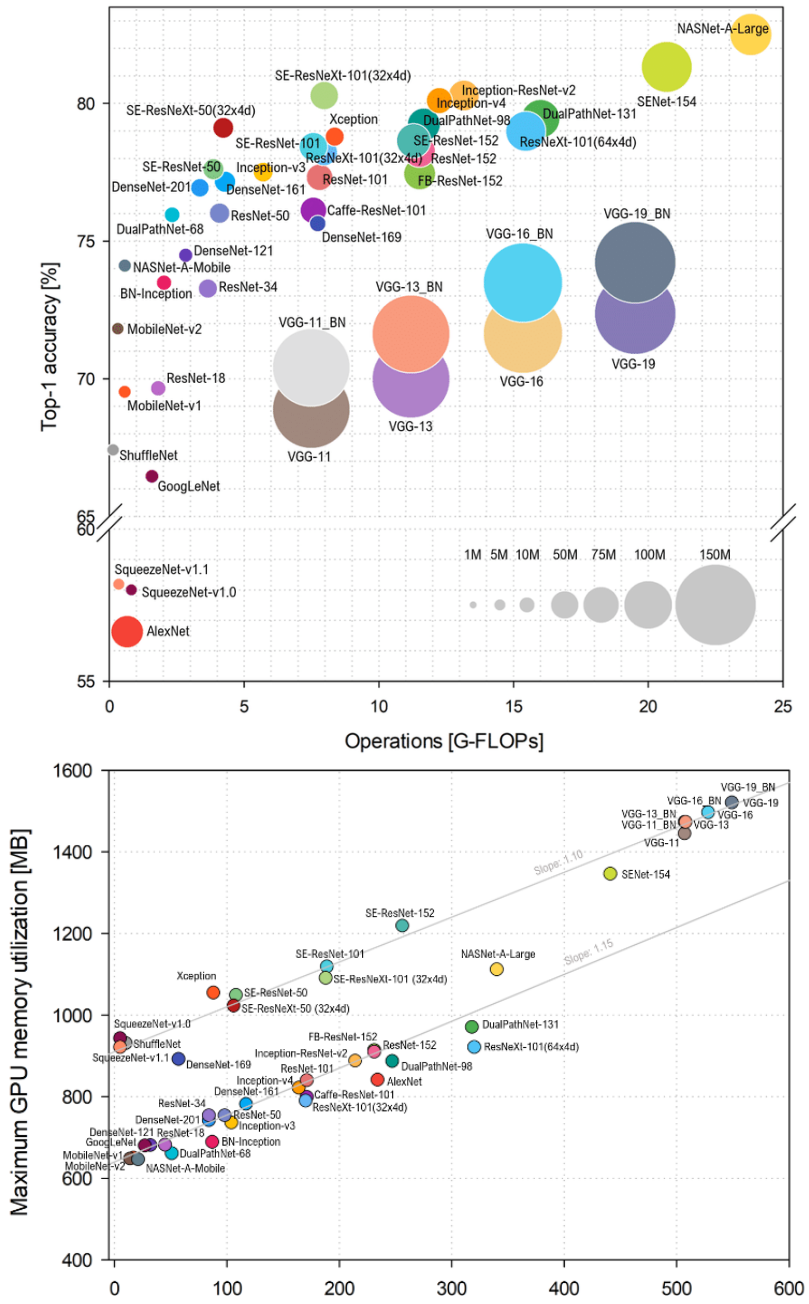

[Krizhevsky et al. NIPS 2012] [Szegedy et al. CVPR 2015] [Simonyan & Zisserman, ICLR 2015] [He et al. CVPR 2016]

```python
import torch
import torch.nn as nn
import torchvision.models as models

# 1. Load pre-trained model
mobilenet = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.DEFAULT)

# 2. Freeze all parameters (feature extractor)
for param in mobilenet.parameters():
    param.requires_grad = False

# 3. Replace the final classification layer
num_features = mobilenet.classifier[1].in_features
num_classes = 10  # Change this to your number of classes
mobilenet.classifier[1] = nn.Linear(num_features, num_classes)
```



# Transfer Learning

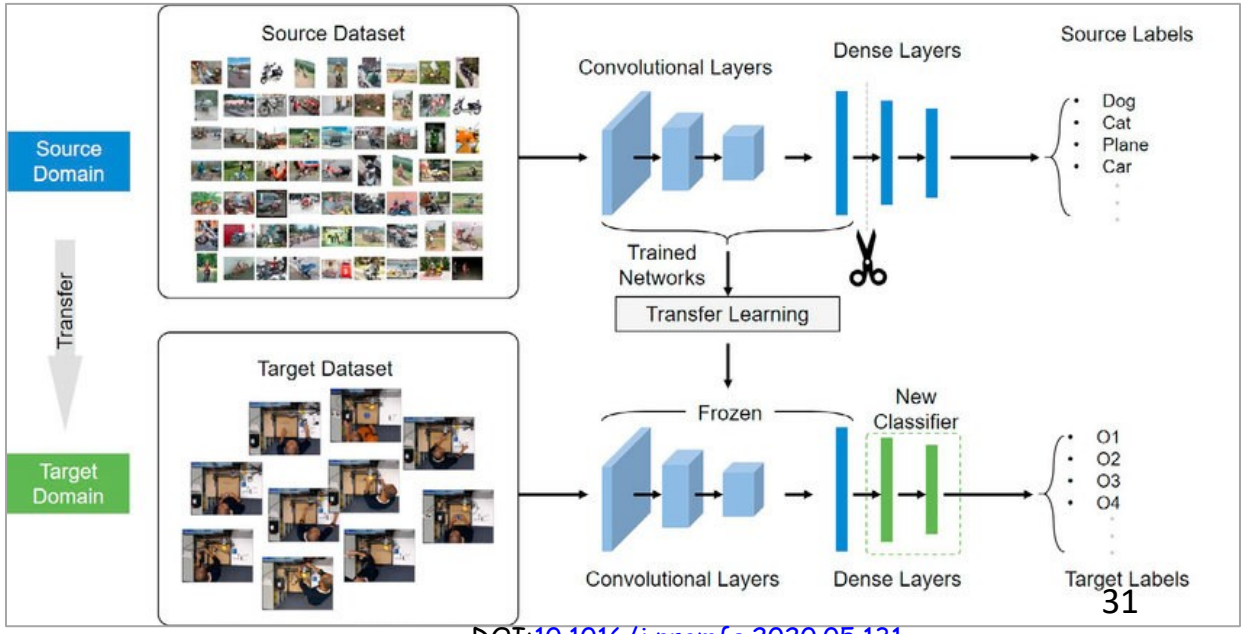
- A model trained on one task (usually with a large dataset) is reused/fine-tuned for a related but different task that may have less data.
- Instead of starting from scratch, leverage knowledge already embedded in a pre-trained model.
  - Example: A CNN trained on ImageNet (millions of images of objects) be adapted to classify medical X-rays (medical data is scarce).
  - Idea: early layers learn generic, reusable features (edges, textures, shapes), while deeper layers learn task-specific representations. We reusing earlier layers and only fine-tuning later ones.



Its advantages:

- Solves data scarcity: reuse trained large models, apply them to small datasets
- Reduces training cost: no need to train deep networks from scratch (expensive data, GPU and energy)
- Improved performance: initializing with pre-trained weights leads to faster convergence and higher accuracy
- Knowledge reuse: features learned in one context (e.g., textures in vision, word embeddings in NLP) often useful across domains

- Pre-training phase: Train a large neural network on a massive, general-purpose dataset (e.g., ImageNet for vision, CommonCrawl for text, Yahoo finance for time series).
- Fine-tuning phase: Take the pre-trained network and freeze some early layers since they capture generic features.
- Replace/Retrain final layers: Adapt higher layers to the new dataset/task.






DOI:10.1016/j.promfg.2020.05.131

# 4.4. Developing CNNs for Time Series Forecasting

## 1D CNN

# Univariate CNN Models

- CNNs can be used to model univariate time series forecasting problems
- CNN model will learn a function that maps a sequence of past observations as input to an output observation.
  1. Data Preparation:
     - the sequence of observations must be transformed into multiple examples from which the model can learn (format suitable for supervised learning).

```
[10 20 30] 40
[20 30 40] 50
[30 40 50] 60
[40 50 60] 70
[50 60 70] 80
[60 70 80] 90
```

```python
# univariate data preparation
from numpy import array
# split a univariate sequence into samples
def split_sequence(sequence, n_steps):
    X, y = list(), list()
    for i in range(len(sequence)):
        # find the end of this pattern
        end_ix = i + n_steps
        # check if we are beyond the sequence
        if end_ix > len(sequence)-1:
            break
        # gather input and output parts of the pattern
        seq_x, seq_y = sequence[i:end_ix], sequence[end_ix]
        X.append(seq_x)
        y.append(seq_y)
    return array(X), array(y)

# define input sequence
raw_seq = [10, 20, 30, 40, 50, 60, 70, 80, 90]
# choose a number of time steps
n_steps = 3
# split into samples
X, y = split_sequence(raw_seq, n_steps)
# summarize the data
for i in range(len(X)):
    print(X[i], y[i])
```

# Univariate CNN Models

2. CNN model
   - A 1D CNN is a CNN model that has a convolutional hidden layer that operates over a 1D sequence (can be followed by perhaps a second convolutional layer in some cases, such as very long input sequences)
   - Then a pooling layer to distill the output of the convolutional layer to the most salient elements.
   - Followed by a fully connected layer (flattening before this layer)

```
# define model
model = Sequential()
model.add(Conv1D(filters=64, kernel_size=2, activation='relu', input_shape=(n_steps,
n_features)))
model.add(MaxPooling1D(pool_size=2))
model.add(Flatten())
model.add(Dense(50, activation='relu'))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
# fit model
model.fit(X, y, epochs=1000, verbose=0)
# demonstrate prediction
x_input = array([70, 80, 90])
x_input = x_input.reshape((1, n_steps, n_features))
yhat = model.predict(x_input, verbose=0)
print(yhat)
```

```
[[101.499664]]
```

*Note*: Neighboring data points, which were produced close together in time, are much more likely to be related than points far apart (convolution finds local patterns that reoccur in the data).

# Multivariate CNN Models

There is more than one observation for each time step:

- Multiple Input Series.
- Multiple Parallel Series.

# Multivariate CNN Models - Multiple Input Series

1. Data Preparation:
   - We have two or more parallel input time series and an output time series that is dependent on the input time series.

```
# define input sequence
in_seq1 = array([10, 20, 30, 40, 50, 60, 70, 80, 90])
in_seq2 = array([15, 25, 35, 45, 55, 65, 75, 85, 95])
out_seq = array([in_seq1[i]+in_seq2[i] for i in range(len(in_seq1))])
```

   - We reshape the dataset.

```
# convert to [rows, columns] structure
in_seq1 = in_seq1.reshape((len(in_seq1), 1))
in_seq2 = in_seq2.reshape((len(in_seq2), 1))
out_seq = out_seq.reshape((len(out_seq), 1))
# horizontally stack columns
dataset = hstack((in_seq1, in_seq2, out_seq))
print(dataset)
```

```
[[ 10  15  25]
 [ 20  25  45]
 [ 30  35  65]
 [ 40  45  85]
 [ 50  55 105]
 [ 60  65 125]
 [ 70  75 145]
 [ 80  85 165]
 [ 90  95 185]]
```

# Multivariate CNN Models - Multiple Input Series

1. Data Preparation:
   - CNNs can support parallel input time series as separate channels, like red, green, and blue components of an image.
   - Therefore, we need to split the data into samples maintaining the order of observations across the two input sequences.

```
# split a multivariate sequence into samples
def split_sequences(sequences, n_steps):
    X, y = list(), list()
    for i in range(len(sequences)):
        # find the end of this pattern
        end_ix = i + n_steps
        # check if we are beyond the dataset
        if end_ix > len(sequences):
            break
        # gather input and output parts of the pattern
        seq_x, seq_y = sequences[i:end_ix, :-1], sequences[end_ix-1, -1]
        X.append(seq_x)
        y.append(seq_y)
    return array(X), array(y)
```

```
n_steps = 3
# convert into input/output
X, y = split_sequences(dataset, n_steps)
print(X.shape, y.shape)
# summarize the data
for i in range(len(X)):
    print(X[i], y[i])
```

```
(7, 3, 2) (7,)
[[10 15]
 [20 25]
 [30 35]] 65
[[20 25]
 [30 35]
 [40 45]] 85
[[30 35]
 [40 45]
 [50 55]] 105
[[40 45]
 [50 55]
 [60 65]] 125
[[50 55]
 [60 65]
 [70 75]] 145
[[60 65]
 [70 75]
 [80 85]] 165
[[70 75]
 [80 85]
 [90 95]] 185
```

# Multivariate CNN Models

2. CNN model
   - A 1D CNN model that has a convolutional hidden layer that operates over a 1D sequence (can be followed by perhaps a second convolutional layer in some cases, such as very long input sequences)
   - Then a pooling layer to distill the output of the convolutional layer to the most salient elements.
   - Followed by a fully connected layer (flattening before this layer)

```
Model: "sequential_3"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv1d_2 (Conv1D)           (None, 2, 64)             192

 max_pooling1d_2 (MaxPoolin  (None, 1, 64)             0
 g1D)

 flatten_2 (Flatten)         (None, 64)                0

 dense_4 (Dense)             (None, 50)                3250

 dense_5 (Dense)             (None, 1)                 51

=================================================================
Total params: 3493 (13.64 KB)
Trainable params: 3493 (13.64 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

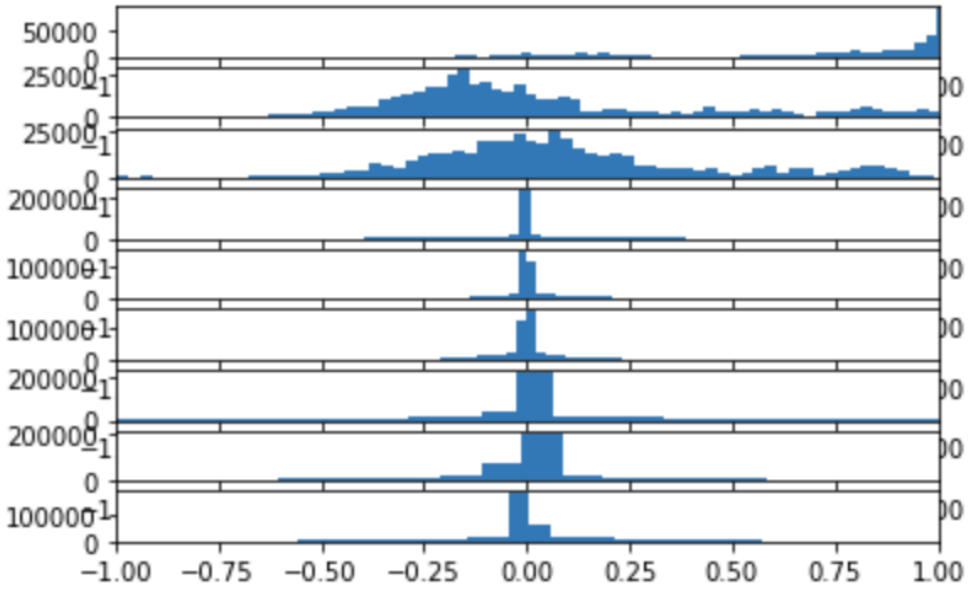
```
# define model
model = Sequential()
model.add(Conv1D(filters=64, kernel_size=2, activation='relu', input_shape=(n_steps,
n_features)))
model.add(MaxPooling1D(pool_size=2))
model.add(Flatten())
model.add(Dense(50, activation='relu'))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
```

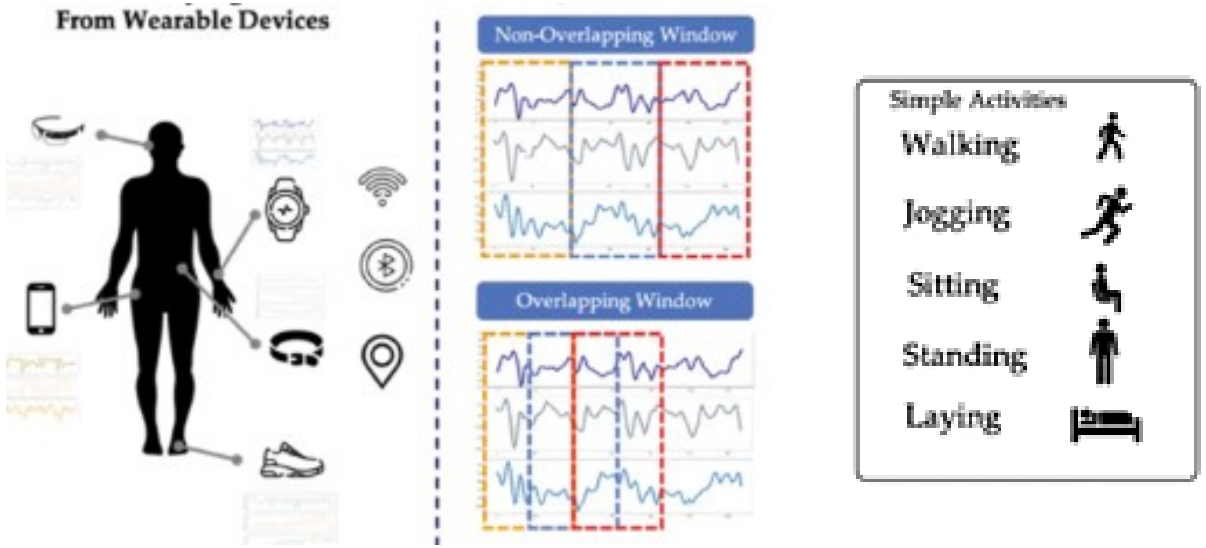
```
# fit model
model.fit([X1, X2], y, epochs=1000, verbose=0)
# demonstrate prediction
x_input = array([[80, 85], [90, 95], [100, 105]])
x1 = x_input[:, 0].reshape((1, n_steps, n_features))
x2 = x_input[:, 1].reshape((1, n_steps, n_features))
yhat = model.predict([x1, x2], verbose=0)
print(yhat)
```

```
[[206.4943]]
```

# Example- UCI HAR dataset:

- A dataset of human action recognition (HAR) including walking, walking upstairs, walking downstairs, sitting, standing, laying.
- The movement data recorded was the x, y, and z accelerometer data (linear acceleration) and gyroscopic data (angular velocity) from the smart phone.
- Each subject performed the sequence of activities twice, once with the device on their left-hand-side and once with the device on their right-hand side.

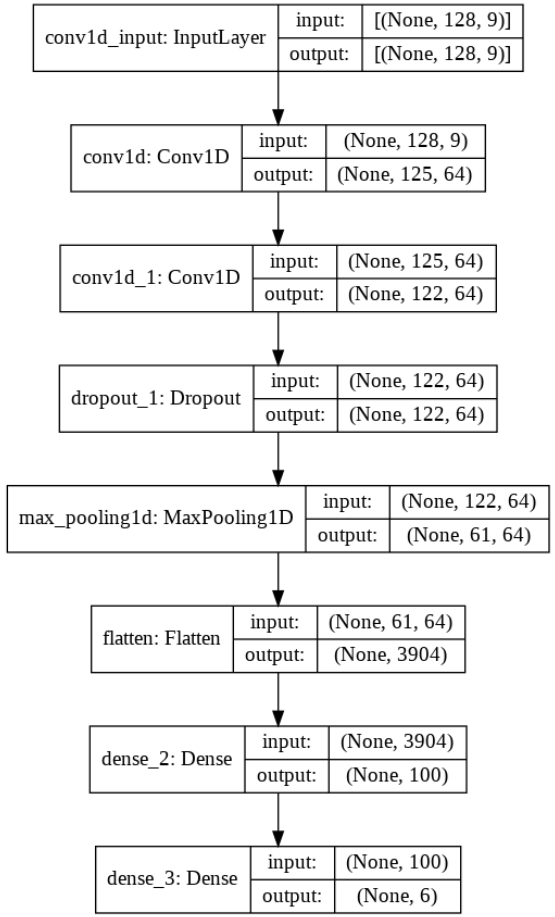



Download dataset from:
https://archive.ics.uci.edu/ml/datasets/Human+Activity+Recognition+Using+Smartphones

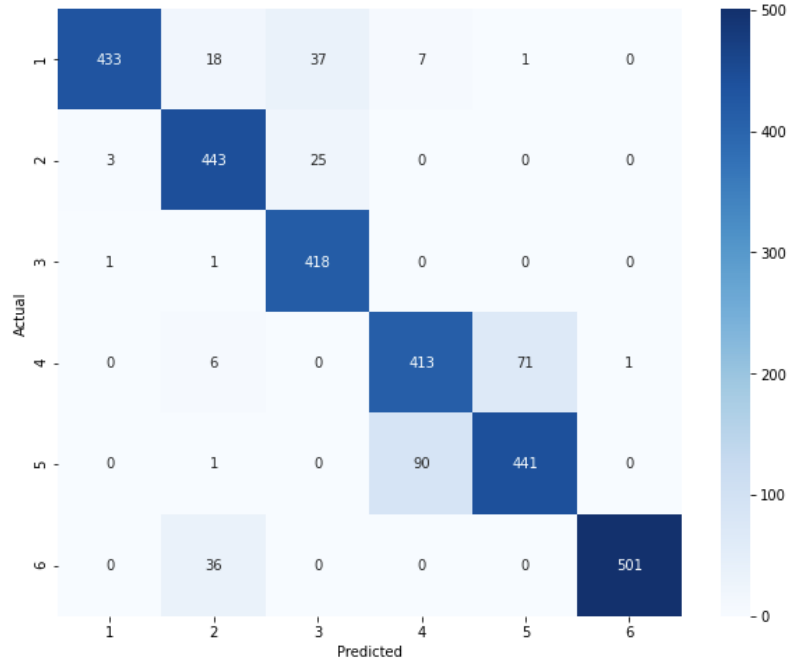
# Example:

- CNN Model




```
n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], trainy.shape[1]
model = Sequential()
model.add(Conv1D(filters=64, kernel_size=4, activation='relu', input_shape=(n_timesteps,n_features)))
model.add(Conv1D(filters=64, kernel_size=4, activation='relu', input_shape=(n_timesteps,n_features)))
model.add(Dropout(0.5))
model.add(MaxPooling1D(pool_size=2))
model.add(Flatten())
model.add(Dense(100, activation='relu'))
model.add(Dense(n_outputs, activation='softmax'))
```

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 1 | 0.99 | 0.87 | 0.93 | 496 |
| 2 | 0.88 | 0.94 | 0.91 | 471 |
| 3 | 0.87 | 1.00 | 0.93 | 420 |
| 4 | 0.81 | 0.84 | 0.83 | 491 |
| 5 | 0.86 | 0.83 | 0.84 | 532 |
| 6 | 1.00 | 0.93 | 0.96 | 537 |
| accuracy | | | 0.90 | 2947 |
| macro avg | 0.90 | 0.90 | 0.90 | 2947 |
| weighted avg | 0.90 | 0.90 | 0.90 | 2947 |



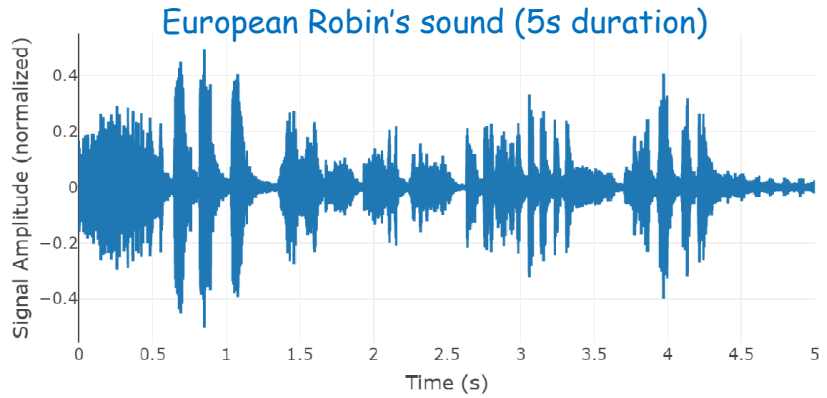
# 4.5. Time-Frequency Analysis for Time Series Data

## 2D CNN

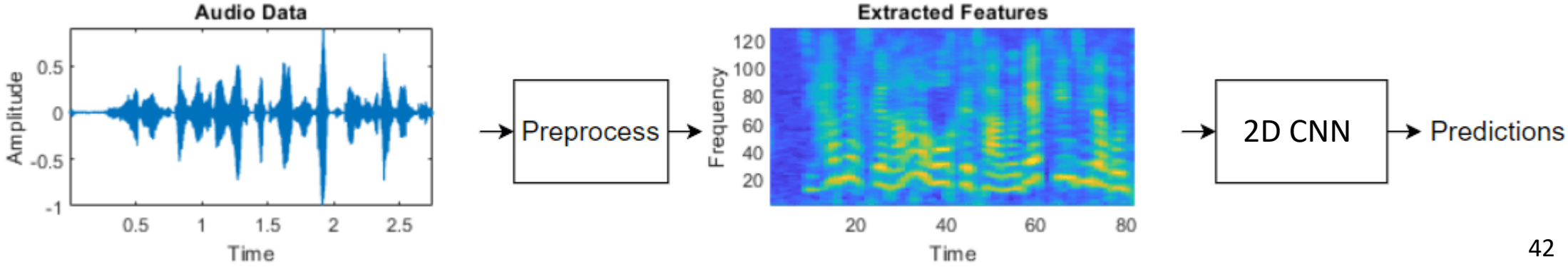
# A Different Solution: 2D CNN for Time Series

## Challenges of time series and limitations of time-domain methods:

- Many real-world signals (e.g., audio, biomedical signals, financial data) are non-stationary and involve localized events and frequency changes over time.
- Capturing "spatial" inter-channel relationships in multi-variate data, handling high-frequency noise, and managing the computational demands of multiple input variables
- Need for learning richer, physically meaningful/interpretable and more structured representations of the raw time series, such as hidden patterns of localized frequency changes.

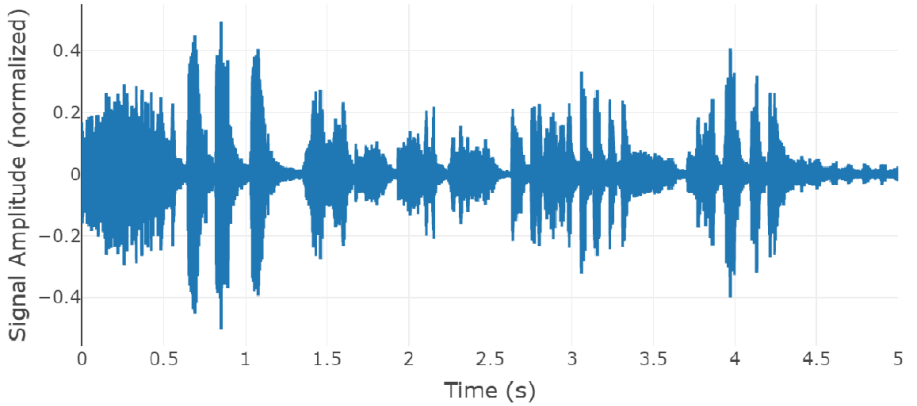


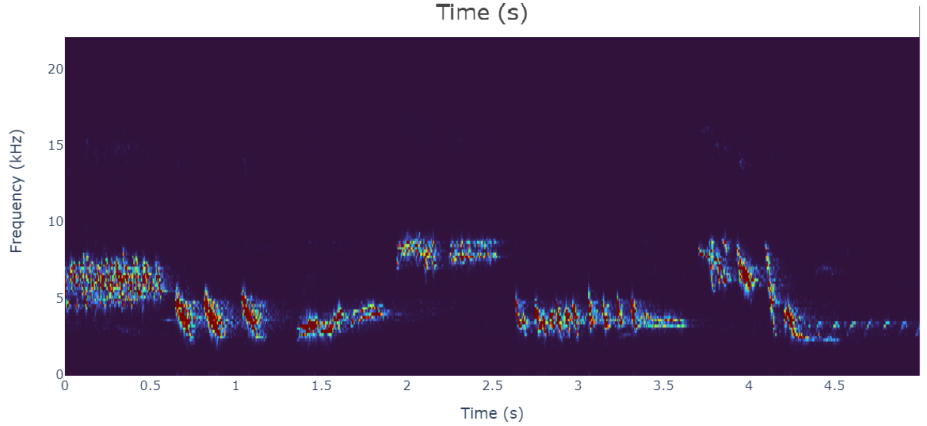

- 2D CNN can also be applied to time series
- Idea: transform 1D time series into a 2D representation (image-like format), then apply 2D CNNs using different techniques such as time-channel matrix (for multivariate analysis where rows are variables and columns are time steps), time-frequency spectrums (Wavelet transform, Short-Time Fourier Transform)



# Frequency and Time-Frequency Domain

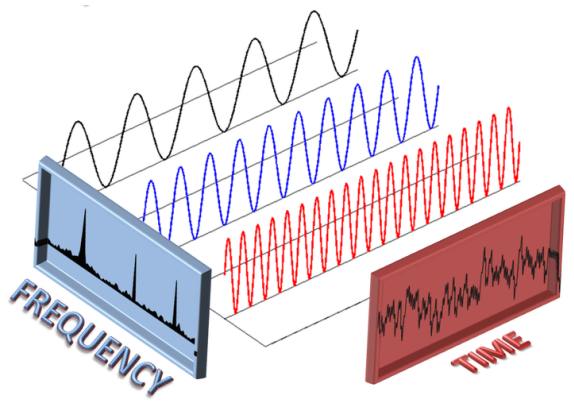
- Transforming a signal from its time domain to frequency domain for time-frequency analysis offers benefits such as:
  - capturing both short-term and long-term patterns
  - improving forecasting accuracy by separating signals from noise,
  - reducing dimensionality
  - revealing hidden patterns and isolating important features (e.g., separating low-frequency components (like trends) from high-frequency components (like noise or details))
  - better analysis and prediction results
  - anomaly detection by identifying deviations in spectral power distributions




For example: in Fourier transform the peaks in the frequency spectrum indicate the most occurring frequencies in the signal.

The location (frequency-value) and height (amplitude) of the peaks in the frequency spectrum then can be used as input for a machine learning model.

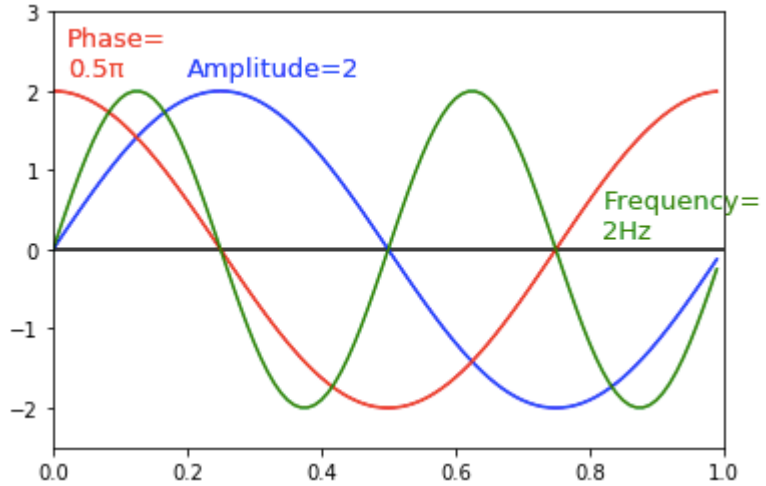


# Fourier Transform (FT)

- A sine function with amplitude (A: the maximum height from its zero base) and frequency (f: the number of complete cycles in a set period (number of complete cycles in 1 sec $\equiv HZ$)

$$y(t) = A\sin(2\pi ft + \varphi) = A\sin(\omega t + \varphi)$$

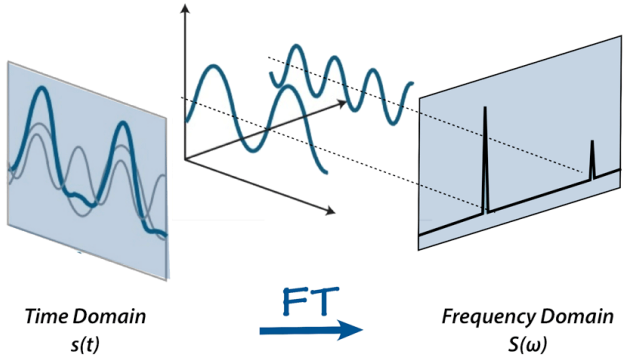
angular frequency  Phase/time shift




- FT/frequency-domain representation decomposes a time-series into its frequencies and amplitude.
- FT decomposes a function into simple sines and cosines of different amplitudes and frequencies.
- But tells you nothing about when the frequencies occur. It works best on signals where the frequency content does not change over time (stationary signals)

$$Y(\omega) = \int_{-\infty}^{\infty} y(t)e^{-i\omega t}dt$$

Complex exponential to extract the frequency components from the signal



# Types of Fourier Transform

- Continuous Fourier Transform (FT): applies to continuous, non-periodic (aperiodic) signals (uses continuous integration in calculation)

$$Y(\omega) = \int_{-\infty}^{\infty} y(t) e^{-i\omega t} dt$$

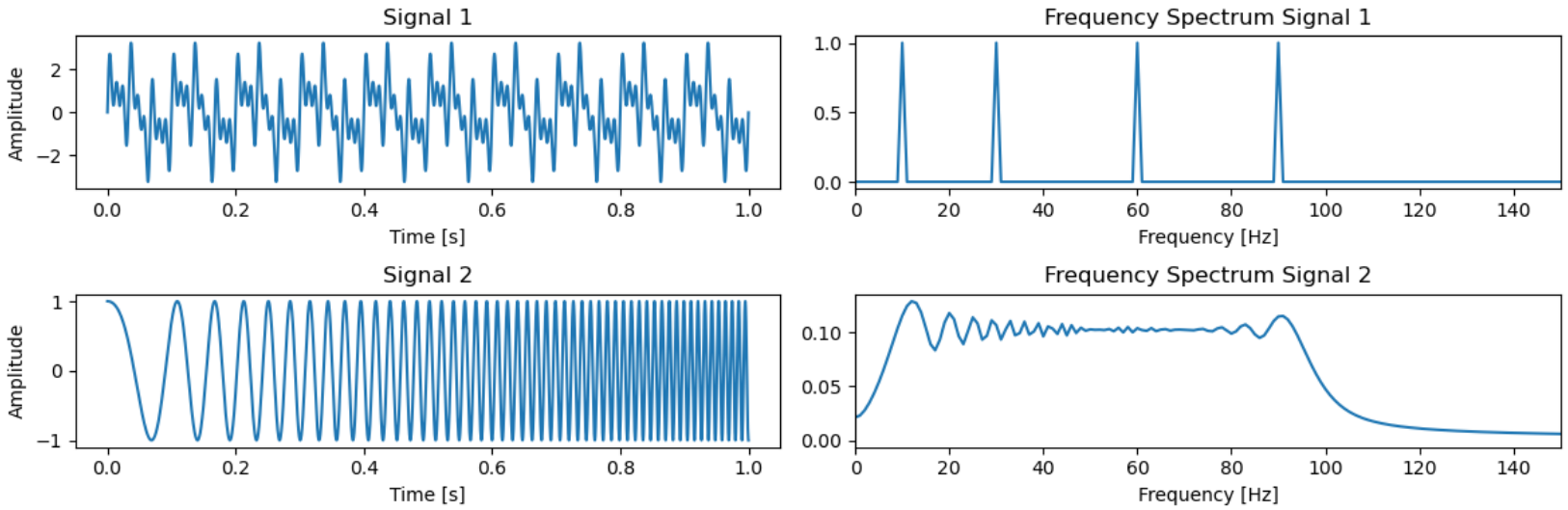
- Discrete Fourier Transform (DFT): works with finite-length, discrete (digital) or sampled sequences (uses discrete summation in calculation)

$$Y[k] = \sum_{n=0}^{N-1} y[n] e^{-i2\pi nk/N}$$

- To perform the DFT quickly we use an efficient algorithm called Fast Fourier Transform (FFT)

# Fourier Transform

- Example:




```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import chirp

def get_fft_values(y, T):
    N = len(y)
    f = np.linspace(0.0, 1.0 / (2.0 * T), N // 2)
    Y = np.fft.fft(y)
    Y = 2.0 / N * np.abs(Y[:N // 2])
    return f, Y
# Parameters
t_n = 1.0
N = 100_000
T = t_n / N
t = np.linspace(0, t_n, N, endpoint=False)

# Signal 1: stationary sinusoids
freqs = [10, 30, 60, 90]
signal1 = sum(np.sin(2 * np.pi * f * t) for f in freqs)

# Signal 2: linear chirp
signal2 = chirp(t, f0=4, f1=99, t1=t_n, method='linear')

# FFTs
f1, S1 = get_fft_values(signal1, T)
f2, S2 = get_fft_values(signal2, T)

# Plot
fig, ax = plt.subplots(2, 2, figsize=(12, 4))

ax[0, 0].plot(t, signal1)
ax[0, 0].set_title('Signal 1')
ax[0, 0].set_xlabel('Time [s]')
ax[0, 0].set_ylabel('Amplitude')

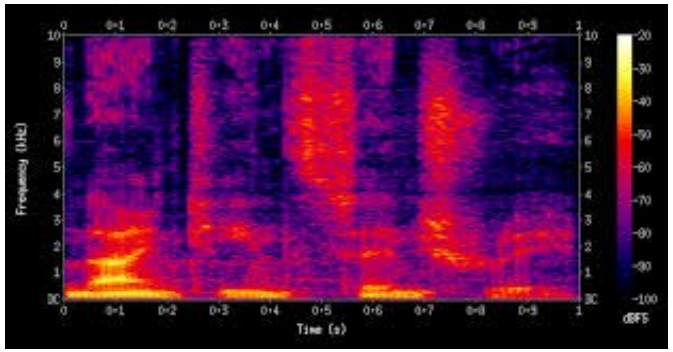
ax[0, 1].plot(f1, S1)
ax[0, 1].set_title('Frequency Spectrum Signal 1')
ax[0, 1].set_xlabel('Frequency [Hz]')
ax[0, 1].set_xlim(0, 150)

ax[1, 0].plot(t, signal2)
ax[1, 0].set_title('Signal 2')
ax[1, 0].set_xlabel('Time [s]')
ax[1, 0].set_ylabel('Amplitude')

ax[1, 1].plot(f2, S2)
ax[1, 1].set_title('Frequency Spectrum Signal 2')
ax[1, 1].set_xlabel('Frequency [Hz]')
ax[1, 1].set_xlim(0, 150)

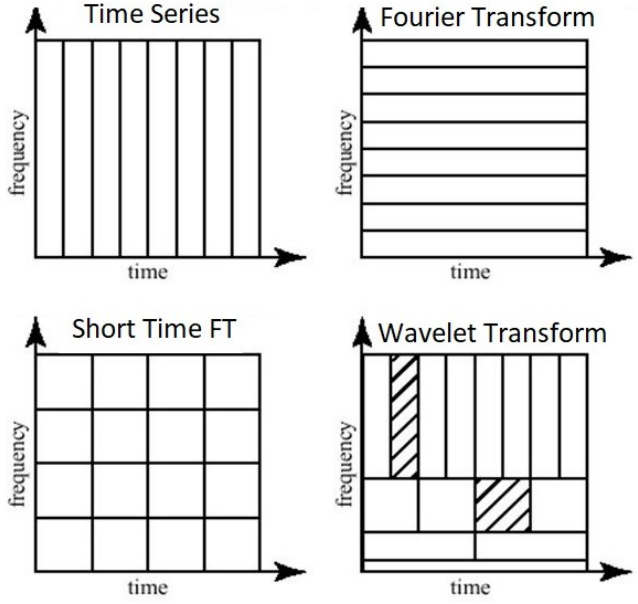
plt.tight_layout()
plt.show()
```

# Short-Time Fourier Transform (STFT)




- A time-frequency technique to analyze how frequency content of a signal changes over time.
  - Segmentation (Windowing): signal is divided into shorter segments of equal length using a "window function" (e.g., Hann, Hamming, Gaussian). The window slides/hops forward through time (typically with overlap to ensure continuity).
  - Fast Fourier Transform (FFT) is computed for each windowed segment (yields a set of frequency components (magnitude and phase) for that specific, short time frame).
  - Results are stacked to form a 2D matrix/image (spectrogram) where time is horizontal axis, frequency is vertical axis, and intensity/colour represents amplitude/energy of frequency component at that specific time.

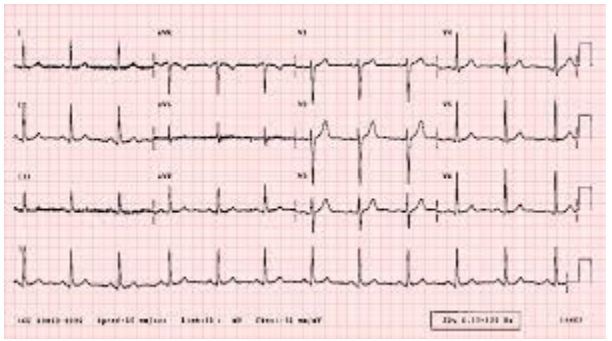
- STFT solves time localization problem in FT.
- Main limitation is the fixed window size (a narrow window gives good time resolution but poor frequency resolution, and vice versa).
- Wavelet Transform improves upon STFT by using a variable-sized window or "multi-resolution" analysis excellent for complex, non-stationary data (short windows for high frequencies/good time localization, and long windows for low frequencies/good frequency localization).



# Wavelet Transform

- A much better approach for analyzing dynamic signals is to use the Wavelet Transform instead of the Fourier Transform.

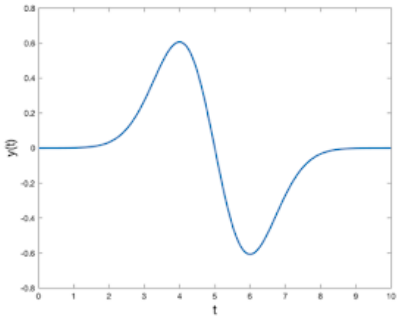
- A major disadvantage of the FT is that it captures only global frequency information (frequencies that persist over an entire signal).
- FT decomposition may not serve all applications (for example, electrocardiogram (ECG), where signals have short intervals of characteristic oscillation).



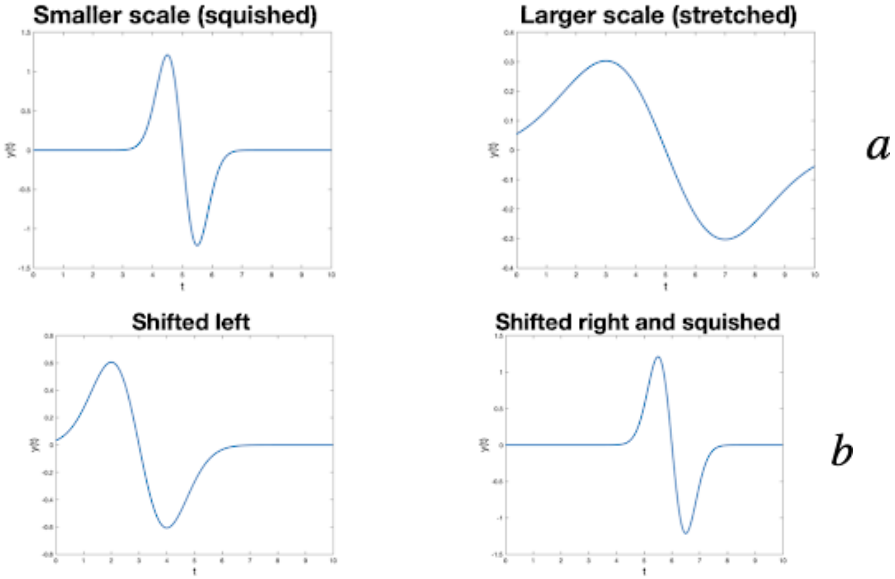
- An alternative approach is the wavelet transform (WT), which decomposes a function into a set of wavelets.

- WT is a very powerful tool for the analysis and classification of time-series and signals.

# What is Wavelet?



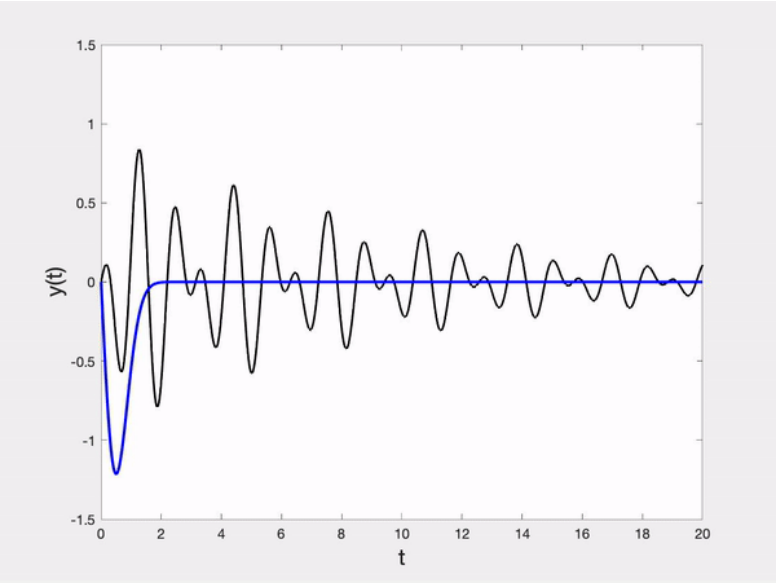
- A wavelet is a wave-like oscillation localized in time
- Wavelets have two basic properties: scale and location.
  - Scale/dilation ($a$) defines how stretched or squished a wavelet is (relates to frequency as defined for waves).
  - Location ($b$) defines where the wavelet is positioned in time (or space).

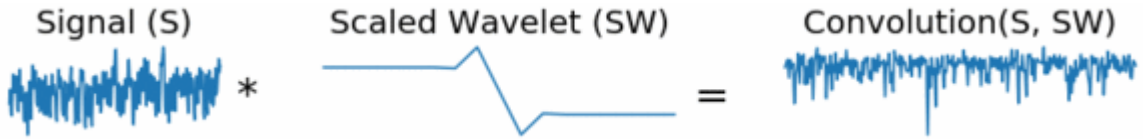



- Wavelet transform can extract local **spectral and temporal information simultaneously**.
- There are a variety of wavelets from which to choose (like Daubechies, Haar, Morlet, Meyer, Symlet and more). Depending on what characteristic shape you're trying to extract from your signal.

# Wavelet Transform

- The basic idea is to compute how much of a wavelet is in a signal for a particular scale and location.
- A signal is convolved with a set of wavelets at a variety of scales.
  - Pick a wavelet of particular scale and slide this wavelet across the entire signal (change location)
  - Multiplication of the wavelet and signal at each time-step gives a coefficient for that wavelet scale at that particular time step.
  - Then change the wavelet scale and repeat the process.



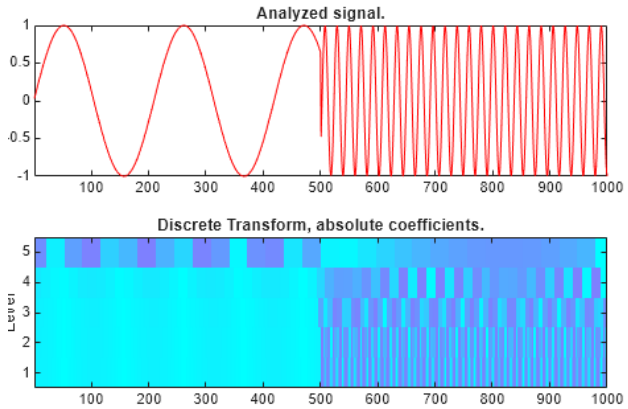



# Types of Wavelet Transform

- The two main types of wavelet transformation are continuous wavelet transformation (CWT) and discrete wavelet transformation (DWT).

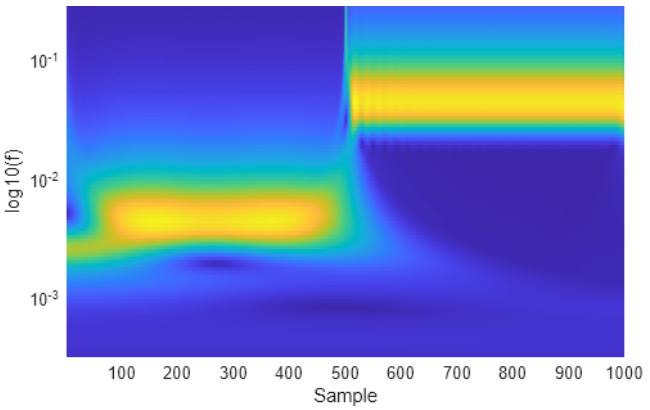
- DWT uses a discrete, non-redundant set of scales, powers of 2 (a=1, 2, 4, … ), and translations, integer values (b=1, 2, 3, … ), for a more efficient, sparse representation.

  - Provides a multi-resolution, hierarchical view of a signal, which is less detailed
  - Faster and more efficient, making it suitable for real-time applications
  - Ideal for signal/image compression, denoising, and efficient data representation.




- CWT uses continuous, overlapping scales and translations (can be an infinite amount of wavelets) for a high-resolution, detailed analysis (for example you can scale the mother wavelet with a factor of 1.3, or 1.31, and 1.311, and 1.3111 etc.).

  - It is computationally intensive but offers greater detail
  - It is often easier to interpret
  - Ideal for **high-fidelity visualization and feature detection**



# Examples of Time-Frequency Transforms' Benefits

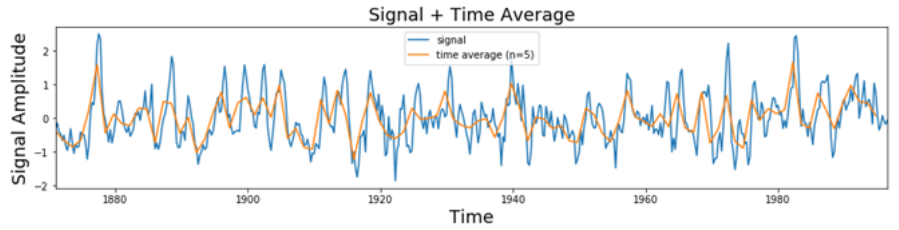
- Using a 2D CNN on a spectrogram/scalogram (output of STFT/CWT, respectively) can offer significant benefits over a 1D CNN on raw time series.
- By leveraging "spatial" correlation between time and frequency

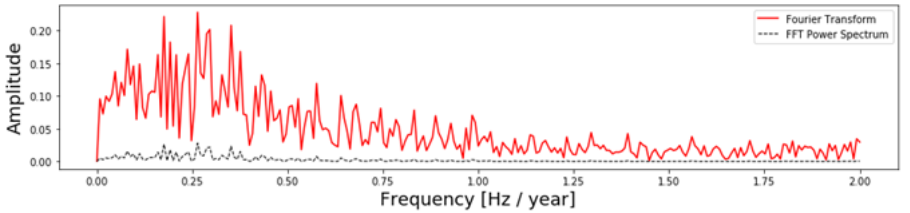
- Because 2D CNNs operate on image data, we can potentially use transfer learning by initializing model with weights pre-trained on large image datasets like ImageNet.

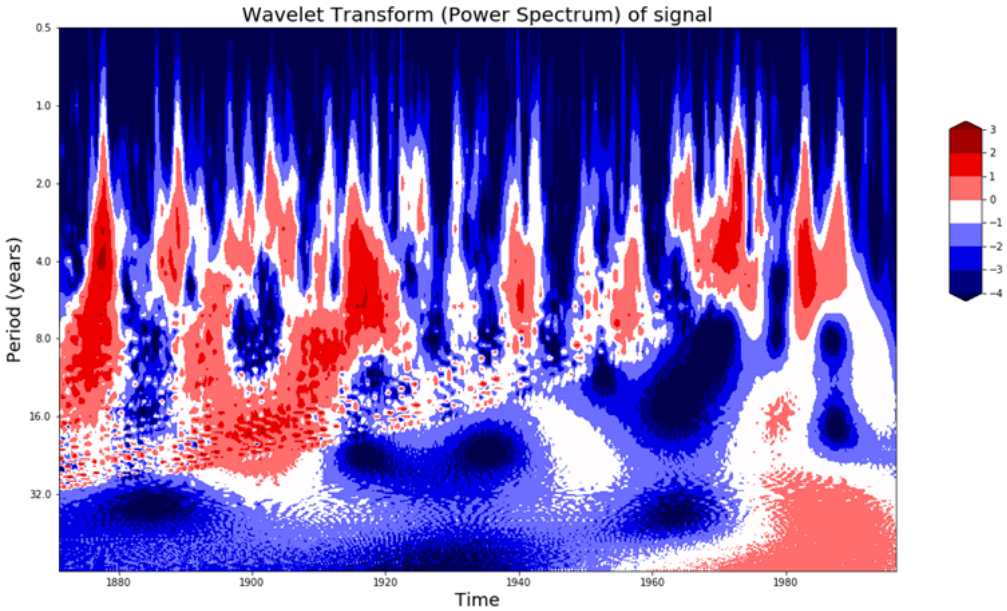
- 2D CNN on spectrograms can outperform 1D models for certain signals, particularly when there is temporal evolution of frequency.

In this Example:

- We can see most of power is concentrated in a 2-8 year period
- An increase in power can also be seen in the Fourier transform around these frequency values. The main difference is that the wavelet transform also gives us temporal information.
- Dynamic behaviour in the signal can be visualized with the CWT but not with FT.



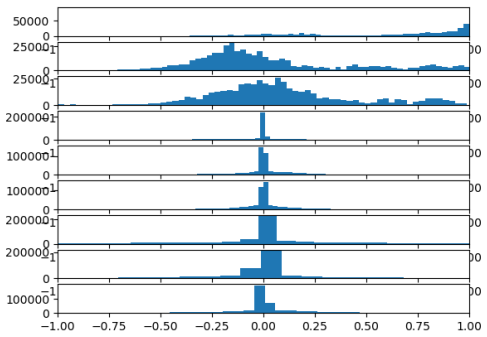



# UCI Example

Ex.1/ The Human Activity Recognition Dataset (UCI-HAR)

An Example for CNN classification using CWT images










```
import pywt
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict, Counter
import keras
from keras.layers import Dense, Flatten
from keras.layers import Conv2D, MaxPooling2D
from keras.models import Sequential
from keras.callbacks import History
```

```
activities_description = {
    1: 'walking',
    2: 'walking upstairs',
    3: 'walking downstairs',
    4: 'sitting',
    5: 'standing',
    6: 'laying'}

def read_signals(filename):
    with open(filename, 'r') as fp:
        data = fp.read().splitlines()
        data = map(lambda x: x.rstrip().lstrip().split(), data)
        data = [list(map(float, line)) for line in data]
    return data

def read_labels(filename):
    with open(filename, 'r') as fp:
        activities = fp.read().splitlines()
        activities = list(map(lambda x: int(x)-1, activities))
    return activities

def randomize(dataset, labels):
    permutation = np.random.permutation(labels.shape[0])
    shuffled_dataset = dataset[permutation, :, :]
    shuffled_labels = labels[permutation]
    return shuffled_dataset, shuffled_labels

INPUT_FOLDER_TRAIN = 'train/InertialSignals/'
INPUT_FOLDER_TEST = 'test/InertialSignals/'
INPUT_FILES_TRAIN = ['body_acc_x_train.txt', 'body_acc_y_train.txt', 'body_acc_z_train.txt',
                     'body_gyro_x_train.txt', 'body_gyro_y_train.txt', 'body_gyro_z_train.txt',
                     'total_acc_x_train.txt', 'total_acc_y_train.txt', 'total_acc_z_train.txt']
INPUT_FILES_TEST = ['body_acc_x_test.txt', 'body_acc_y_test.txt', 'body_acc_z_test.txt',
                    'body_gyro_x_test.txt', 'body_gyro_y_test.txt', 'body_gyro_z_test.txt',
                    'total_acc_x_test.txt', 'total_acc_y_test.txt', 'total_acc_z_test.txt']
LABELFILE_TRAIN = 'train/y_train.txt'
LABELFILE_TEST = 'test/y_test.txt'
train_signals, test_signals = [], []

for input_file in INPUT_FILES_TRAIN:
    signal = read_signals(INPUT_FOLDER_TRAIN + input_file)
    train_signals.append(signal)
train_signals = np.transpose(np.array(train_signals), (1, 2, 0))

for input_file in INPUT_FILES_TEST:
    signal = read_signals(INPUT_FOLDER_TEST + input_file)
    test_signals.append(signal)
test_signals = np.transpose(np.array(test_signals), (1, 2, 0))

train_labels = read_labels(LABELFILE_TRAIN)
test_labels = read_labels(LABELFILE_TEST)

[no_signals_train, no_steps_train, no_components_train] = np.shape(train_signals)
[no_signals_test, no_steps_test, no_components_test] = np.shape(test_signals)
no_labels = len(np.unique(train_labels[:]))

print("The train dataset contains {} signals, each one of length {} and {} components "
      .format(no_signals_train, no_steps_train, no_components_train))
print("The test dataset contains {} signals, each one of length {} and {} components "
      .format(no_signals_test, no_steps_test, no_components_test))
print("The train dataset contains {} labels, with the following distribution:\n {}"
      .format(np.shape(train_labels)[0], Counter(train_labels[:])))
print("The test dataset contains {} labels, with the following distribution:\n {}"
      .format(np.shape(test_labels)[0], Counter(test_labels[:])))

uci_har_signals_train, uci_har_labels_train = randomize(train_signals, np.array(train_labels))
uci_har_signals_test, uci_har_labels_test = randomize(test_signals, np.array(test_labels))
```

# UCI Example

CNN classification using CWT images:

- Use 5000 signals in our training set and 500 signals in our test set

```
scales = range(1,128)
waveletname = 'morl'
train_size = 5000
train_data_cwt = np.ndarray(shape=(train_size, 127, 127, 9))

for ii in range(0,train_size):
    if ii % 1000 == 0:
        print(ii)
    for jj in range(0,9):
        signal = uci_har_signals_train[ii, :, jj]
        coeff, freq = pywt.cwt(signal, scales, waveletname, 1)
        coeff_ = coeff[:,:127]
        train_data_cwt[ii, :, :, jj] = coeff_

test_size = 500
test_data_cwt = np.ndarray(shape=(test_size, 127, 127, 9))
for ii in range(0,test_size):
    if ii % 100 == 0:
        print(ii)
    for jj in range(0,9):
        signal = uci_har_signals_test[ii, :, jj]
        coeff, freq = pywt.cwt(signal, scales, waveletname, 1)
        coeff_ = coeff[:,:127]
        test_data_cwt[ii, :, :, jj] = coeff_
```

```
def show_signal_and_cwt(idx=0, split='train', channel=0):
    if split == 'train':
        signal = uci_har_signals_train[idx, :, channel]     # (128,)
        img    = train_data_cwt[idx, :, :, channel]         # (127,127)
        label  = uci_har_labels_train[idx]
    else:
        signal = uci_har_signals_test[idx, :, channel]
        img    = test_data_cwt[idx, :, :, channel]
        label  = uci_har_labels_test[idx]

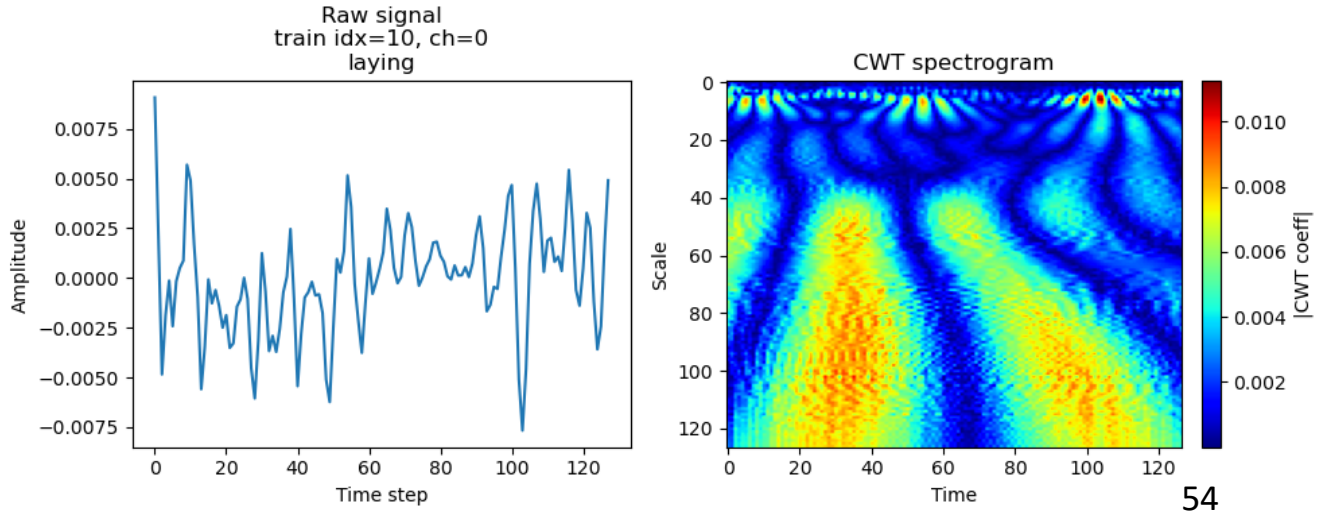
    activity_name = activities_description[label + 1]

    plt.figure(figsize=(10, 4))

    # raw time series
    plt.subplot(1, 2, 1)
    plt.plot(signal)
    plt.title(f"Raw signal\n{split} idx={idx}, ch={channel}\n{activity_name}")
    plt.xlabel("Time step")
    plt.ylabel("Amplitude")

    # CWT spectrogram
    plt.subplot(1, 2, 2)
    plt.imshow(np.abs(img), aspect='auto', cmap='jet', origin='upper')
    plt.title("CWT spectrogram")
    plt.xlabel("Time")
    plt.ylabel("Scale")
    plt.colorbar(label='|CWT coeff|', fraction=0.046, pad=0.04)

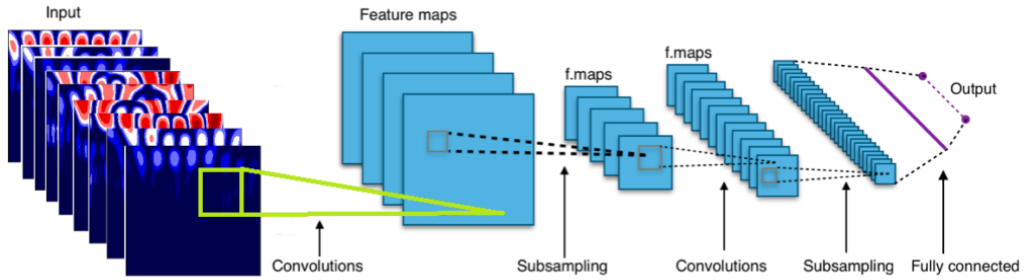
    plt.tight_layout()
    plt.show()
```



# UCI Example

CNN classification using CWT images:

- Train CNN
- Accuracy of 94% (higher than 1D CNN we showed before).




```
x_train = train_data_cwt
y_train = list(uci_har_labels_train[:train_size])
x_test = test_data_cwt
y_test = list(uci_har_labels_test[:test_size])
img_x = 127
img_y = 127
img_z = 9
num_classes = 6

batch_size = 16
epochs = 10

# reshape the data into a 4D tensor
input_shape = (img_x, img_y, img_z)
# convert the data to the right type
#x_train = x_train.reshape(x_train.shape[0], img_x, img_y, img_z)
#x_test = x_test.reshape(x_test.shape[0], img_x, img_y, img_z)
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')

print('x_train shape:', x_train.shape)
print(x_train.shape[0], 'train samples')
print(x_test.shape[0], 'test samples')

# convert class vectors to binary class matrices - this is for use in the
# categorical_crossentropy loss below
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)
```

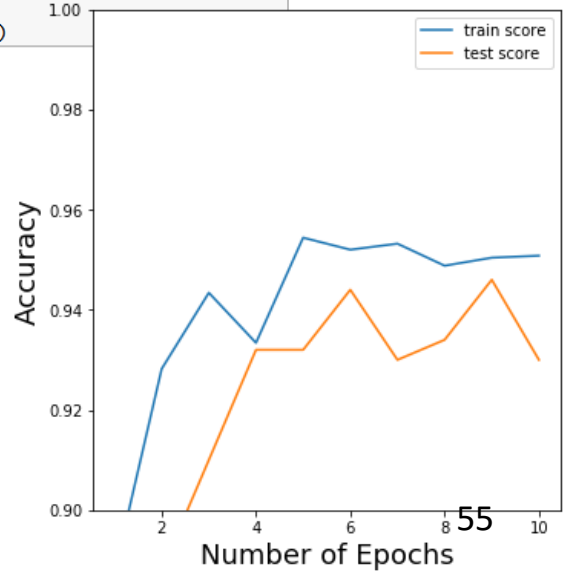
```
x_train shape: (5000, 127, 127, 9)
5000 train samples
500 test samples
```

```
model = Sequential()
model.add(Conv2D(32, kernel_size=(5, 5), strides=(1, 1), activation='relu', input_shape=input_shape))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))
model.add(Conv2D(64, (5, 5), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(1000, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))

model.compile(loss=keras.losses.categorical_crossentropy,
              optimizer=keras.optimizers.Adam(),
              metrics=['accuracy'])

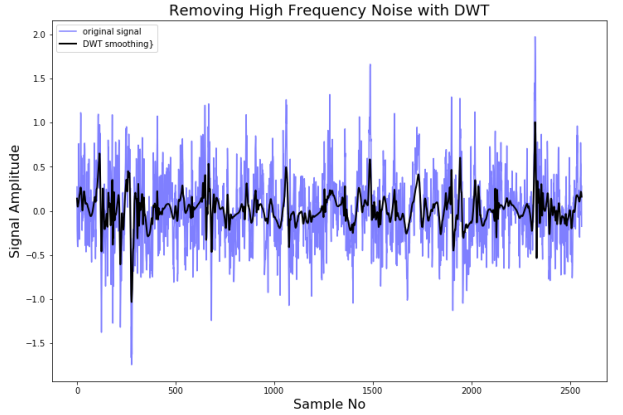
model.fit(x_train, y_train, batch_size=batch_size,
          epochs=epochs, verbose=1,
          validation_data=(x_test, y_test),
          callbacks=[history])

train_score = model.evaluate(x_train, y_train, verbose=0)
print('Train loss: {}, Train accuracy: {}'.format(train_score[0], train_score[1]))
test_score = model.evaluate(x_test, y_test, verbose=0)
print('Test loss: {}, Test accuracy: {}'.format(test_score[0], test_score[1]))
```

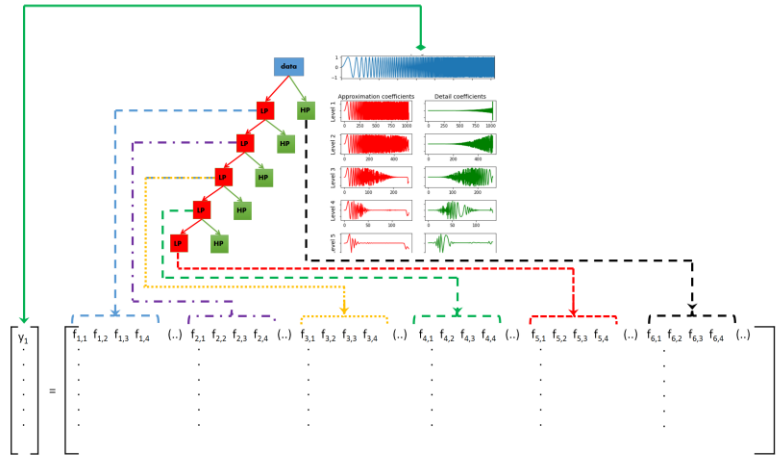


# Examples of Time-Frequency Transforms' Benefits

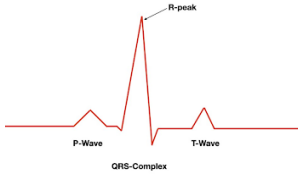
- Ex.2/ Deconstructing a clean signal using the DWT:
  - Deconstruct a signal into the approximation (low pass) and detail (high pass) coefficients
  - Reconstruct while we leave out some detail coefficients (filtered out high frequency part of the spectrum/noise)

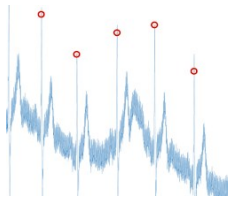


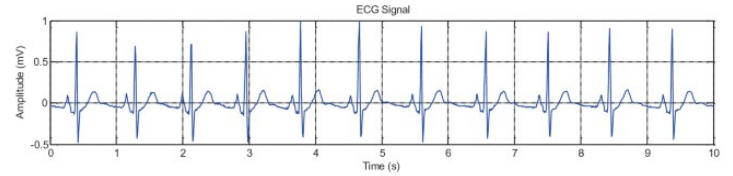

- Ex.3/ Feature Engineering using the DWT:
  - Split a signal into different frequency sub-bands
  - Generate features from each of the sub-band and use the collection of features as an input for a classifier

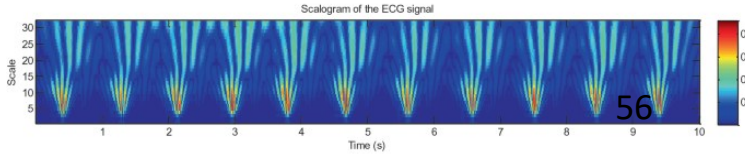



- Ex. 4/ Feature Engineering using the DWT:
  - DWT can detect R-peaks from an electrocardiogram (ECG).
  - R-peaks are highest peaks in ECG corresponding to contraction of the ventricles and expansion of the atria.
  - Detecting R-peaks helps in computing heart rate variability (HRV).
  - Simple peak-finding algorithms fail to generalize on raw data
  - DWT makes it much easier for our peak finder function





# References


- M. Aqil, "Continuous Wavelet Analysis and Extraction of ECG Features", pp 51-68, 2021.
- J. Kaub, towardsdatascience, 2023.
- S. Talebi, Fourier vs. Wavelet Transform, 2025.
- P. Kalhara, TreeSpirit, 2017.
- https://mriquestions.com/
- https://www.mathworks.com/help/wavelet.
- https://github.com/taspinar/siml/tree/master.
- https://ela.kpi.ua/